LegalOrbis

Documentación técnica completa

Generado: 08/06/2026

Overview

LegalOrbis is the official corporate web platform for **LegalOrbis**, a multidisciplinary law firm based in Madrid CLAUDE.md:3-5. The application serves as a high-performance, SEO-optimized digital storefront showcasing the firm's expertise in Criminal, Civil, Labor, and Penitentiary law app/page.tsx:7-22.

The codebase is designed as a static-first corporate site leveraging modern React patterns, dynamic component loading for performance, and a robust automated asset pipeline.

Tech Stack

The project is built on a modern frontend stack focused on performance, type safety, and fluid animations.

Category	Technology
Framework	Next.js 16 (App Router) package.json:22
Runtime/Language	Node.js 20+, TypeScript 5 package.json:30,40
UI / Styling	Tailwind CSS 4, Radix UI, Lucide React package.json:14-21,36
Animations	Framer Motion, Embla Carousel package.json:19-20
Deployment	Vercel CLAUDE.md:35

Sources: package.json:1-42, CLAUDE.md:7-9

System Architecture

The following diagram illustrates how the core subsystems interact, bridging the Natural Language concepts to the specific Code Entities.

Code Entity Relationship

```
graph TD
    subgraph "Routing & Pages"
        A["app/page.tsx"] -->|"Home"| B["app/layout.tsx"]
        C["app/areas-juridicas/[slug]/page.tsx"] -->|"Dynamic Routes"| B
    end

    subgraph "Data & Logic"
        D["lib/data/areas-juridicas.ts"] -->|"Content Source"| C
        E["lib/seo/metadata.ts"] -->|"SEO Generation"| A
        E --> C
    end

    subgraph "UI Layer"
        F["components/header.tsx"]
        G["components/hero-carousel.tsx"]
        H["components/ui/"]
    end

    A --> F
    A --> G
    G --> H
```

Sources: app/page.tsx:1-56, CLAUDE.md:21-29

Major Subsystems

1. Developer Onboarding

The project uses standard `npm` scripts for development and maintenance. A specialized `generate-favicons` script is included to manage brand assets via Sharp and `to-ico`. * For setup instructions, see **Getting Started**.

2. Project Organization

The repository follows the Next.js App Router convention with a clear separation between page logic (`app/`), reusable components (`components/`), and business logic/data (`lib/`). * For a deep dive into the directory layout, see **Project Structure**.

3. Dynamic Content Engine

The site's primary content (Legal Areas) is driven by a centralized data model in `lib/data/`. This data populates both the UI components and the SEO metadata generators dynamically. * **Key Data Structure:** `areasData` constant in `lib/data/areas-juridicas.ts`. * **Metadata:** Managed via `generatePageMetadata` and `generateAreaMetadata` app/page.tsx:7-24.

4. CI/CD and Infrastructure

The project includes a GitHub Actions workflow for automated linting and building on every push to `dev` or `main` branches. * **CI Workflow:** `.github/workflows/ci.yml` ci.yml:1-31. * **Dependency Management:** Automated via Renovate renovate.json:1-25.

Infrastructure to Code Mapping

```
graph LR
    subgraph "Infrastructure (Vercel/GitHub)"
        V["vercel.json"]
        CI[".github/workflows/ci.yml"]
    end

    subgraph "Code Logic"
        NC["next.config.ts"]
        SF["scripts/generate-favicons.ts"]
    end

    V -->|"Configures"| NC
    CI -->|"Executes"| SF
```

Sources: vercel.json:1-20, next.config.ts:1-50, package.json:11

Child Sections

- **Getting Started**: Prerequisites, installation, and local development workflow.
- **Project Structure**: Detailed explanation of the directory layout and Next.js conventions.

Getting Started

This page provides the necessary information for developers to set up their local environment, install dependencies, and understand the core development workflow for the LegalOrbis project.

Overview

LegalOrbis is a corporate web application built with **Next.js 16** and **React 19** CLAUDE.md:9-9. It serves as the digital presence for a legal firm, utilizing a modern stack that includes **Tailwind CSS 4**, **Framer Motion** for animations, and **Radix UI** primitives package.json:13-26.

Prerequisites

To contribute to this project, ensure your environment meets the following requirements:

- **Node.js**: Version 20 or higher is required package-lock.json:47-47, .github/workflows/ci.yml:20-20.
- **Package Manager**: `npm` (version included with Node 20).
- **Operating System**: Cross-platform (Linux/macOS/Windows).

Installation

Follow these steps to initialize the project locally:

1. **Clone the repository**: `bash git clone https://github.com/gestionDP/LegalOrbis.git cd LegalOrbis`
2. **Install dependencies**: The project uses a standard `package-lock.json` to ensure deterministic builds. `bash npm install` Sources: CLAUDE.md:14-14, package-lock.json:1-6

Environment Setup

The application uses TypeScript path aliasing to simplify imports. The `@/*` alias points to the root directory, allowing for clean imports across the `app/`, `components/`, and `lib/` folders tsconfig.json:25-29.

TypeScript Configuration

The project targets `ES2017` and uses the `bundler` module resolution strategy tsconfig.json:3-15. It includes specific type definitions for third-party libraries like `to-ico` types/to-ico.d.ts:1-10.

Available Scripts

The following scripts are defined in `package.json` to manage the development lifecycle:

Script	Command	Description
`dev`	`next dev`	Starts the Next.js development server.
`dev:turbo`	`next dev --turbopack`	Starts the development server using the Turbopack engine for faster HMR.
`build`	`next build`	Compiles the application for production.
`start`	`next start`	Starts the production server (requires `npm run build`).
`lint`	`eslint`	Runs ESLint to check for code quality and style issues.
`generate-favicons`	`tsx scripts/generate-favicons.ts`	Executes the favicon generation pipeline using Sharp and to-ico.

Sources: package.json:5-12, CLAUDE.md:11-19

Local Development Workflow

The following diagram illustrates the relationship between the local development environment and the automated CI/CD pipeline.

Development to Deployment Flow

```
graph TD
    subgraph "Local_Development"
        A["npm_run_dev"] --> B["Next.js_Dev_Server"]
        C["npm_run_generate-favicons"] --> D["public_Assets"]
        E["npm_run_lint"] --> F["ESLint_Checks"]
    end

    subgraph "Version_Control"
        G["Git_Push"] --> H["GitHub_Repository"]
    end

    subgraph "CI_Workflow"
        H --> I[".github/workflows/ci.yml"]
        I --> J["npm_ci"]
        J --> K["npm_run_lint"]
        K --> L["npm_run_build"]
    end

    subgraph "Deployment"
        L --> M["Vercel_Edge_Network"]
    end

    B -.-> G
    F -.-> G
```

Sources: package.json:5-12, .github/workflows/ci.yml:1-31, CLAUDE.md:31-35

Asset Generation

A specialized script is used to maintain brand assets. The `generate-favicons` script processes a source SVG to create a full suite of icons required for various browsers and devices.

- **Input**: `public/favicon.svg` (assumed source).
- **Processing**: Uses `sharp` for image manipulation and `to-ico` for `.ico` packaging package.json:35-37.

- **Output**: Generates multiple sizes including 16x16, 32x32, 48x48, 96x96, and 180x180 types/to-ico.d.ts:2-7.

Automated Maintenance

The repository uses **Renovate Bot** to keep dependencies up to date. * **Schedule**: Runs every Monday before 9 AM (Europe/Madrid) renovate.json:4-5. * **Automerge**: Patch updates are automatically merged if they pass CI renovate.json:9-11. * **Groupings**: `Next.js` and `React` updates are grouped to ensure compatibility renovate.json:17-24.

Dependency Relationship Mapping

```
graph LR
    subgraph "Core_Framework"
        "next"["next@16.1.1"]
        "react"["react@19.1.0"]
        "react-dom"["react-dom@19.1.0"]
    end

    subgraph "UI_Styling"
        "tailwindcss"["tailwindcss@4"]
        "framer-motion"["framer-motion@12.23.24"]
        "lucide-react"["lucide-react@0.546.0"]
    end

    subgraph "Radix_Primitives"
        "radix-accordion"["@radix-ui/react-accordion"]
        "radix-dialog"["@radix-ui/react-dialog"]
    end

    "next" --> "react"
    "next" --> "react-dom"
    "UI_Styling" -.-> "next"
    "Radix_Primitives" -.-> "react"
```

Sources: package.json:13-26, package.json:36-36, renovate.json:17-24

Project Structure

The LegalOrbis codebase is organized following the **Next.js App Router** architecture, optimized for a high-performance corporate legal website. It utilizes a modular structure where concerns are strictly separated between routing, UI components, data models, and SEO utilities.

Directory Overview

The project structure is designed to support static generation with dynamic client-side interactivity (Framer Motion, Embla Carousel) and robust SEO via structured data schemas.

Directory	Purpose	Key Contents
`app/`	Routing & Page Layouts	Routes, `layout.tsx`, `not-found.tsx`, and `sitemap.ts`.
`components/`	React Components	Section-level components (e.g., `HeroCarousel`) and UI primitives.
`hooks/`	Custom React Hooks	Reusable logic like `useContactForm`.
`lib/`	Core Logic & Data	Practice area data, SEO metadata generators, and utility functions.
`scripts/`	Build-time Scripts	Favicon generation and asset processing.
`types/`	TypeScript Definitions	Shared interfaces and third-party type augmentations.
`public/`	Static Assets	Images, manifest, and generated favicons.

Sources: CLAUDE.md:21-29, app/layout.tsx:1-69

Routing & Page Architecture (`app/`)

LegalOrbis uses the **Next.js App Router** conventions. The root of the application is defined in `app/layout.tsx`, which wraps all pages with global providers, fonts, and base SEO configurations.

Root Layout and Global Config

The `RootLayout` component in `app/layout.tsx` serves as the entry point for the DOM structure. It initializes: - **Fonts:** Geist Sans and Geist Mono via `next/font/google` app/layout.tsx:11-23. - **SEO Metadata:** Base metadata generated via `generateBaseMetadata()` app/layout.tsx:25. - **Structured Data:** Injects `Organization` and `LocalBusiness` JSON-LD schemas into the `<head>` app/layout.tsx:32-56.

Error Handling

The `app/not-found.tsx` file provides a custom 404 experience. It uses a `main` container with a gradient background and provides navigation buttons to return to the home page or practice areas app/not-found.tsx:16-47. It explicitly sets `robots: { index: false }` to prevent search engines from indexing error pages app/not-found.tsx:10-13.

Request Flow Diagram

The following diagram illustrates how a request is handled by the `app/` directory and which code entities are invoked.

Title: Next.js App Router Request Flow

```
graph TD
    subgraph "Request_Processing"
        URL["Browser URL Request"] --> Router["Next.js App Router"]
        Router --> Layout["app/layout.tsx (RootLayout)"]
    end

    subgraph "Page_Resolution"
        Layout --> Home["app/page.tsx (Home)"]
        Layout --> AreaDetail["app/areas-juridicas/[slug]/page.tsx"]
        Layout --> NotFound["app/not-found.tsx (NotFound)"]
```

```
    end

    subgraph "SEO_&_Data_Injection"
        Home --> MetaHome["generatePageMetadata()"]
        AreaDetail --> MetaArea["generateAreaMetadata()"]
        Layout --> SchemaOrg["generateOrganizationSchema()"]
        Layout --> SchemaBiz["generateLocalBusinessSchema()"]
    end
```

Sources: app/layout.tsx:27-68, app/page.tsx:7-24, app/not-found.tsx:16-49

Component Organization (`components/`)

Components are divided into two distinct layers to maximize reusability and maintainability:

1. **Section Components:** Large, page-specific blocks (e.g., `HeroCarousel`, `QuienesSomos`, `AreasJuridicas`) app/page.tsx:47-53.
2. **UI Primitives:** Atomic components found in `components/ui/` (e.g., `Button`, `Card`, `Carousel`) app/not-found.tsx:5.

Dynamic Loading Strategy

To optimize the Initial Page Load, the Home Page (`app/page.tsx`) uses `next/dynamic` to lazy-load sections below the fold, such as `QuienesSomos`, `VideoBanner`, and `AreasJuridicas` app/page.tsx:26-42.

Title: Component Hierarchy and Data Flow

```
graph BT
    subgraph "UI_Primitives (components/ui/)"
        Button["Button.tsx"]
        Card["Card.tsx"]
        Sheet["Sheet.tsx"]
    end

    subgraph "Section_Components (components/)"
        Header["Header.tsx"]
        Hero["HeroCarousel.tsx"]
        Areas["AreasJuridicas.tsx"]
```

```
        Footer["Footer.tsx"]
    end

    subgraph "Pages (app/)"
        HomePage["page.tsx"]
    end

    Button --> Header
    Button --> Areas
    Card --> Areas
    Header --> HomePage
    Hero --> HomePage
    Areas --> HomePage
    Footer --> HomePage
```

Sources: app/page.tsx:1-42, app/not-found.tsx:3-5

Data and Logic (`lib/`)

The `lib/` directory contains the "brain" of the application, separating business data from UI logic.

- **`lib/data/`** : Contains `areas-juridicas.ts` , the single source of truth for practice area content, FAQs, and slugs.
- **`lib/seo/`** : Houses metadata and schema generators.
  - `metadata.ts` : Logic for `generateBaseMetadata` , `generatePageMetadata` , and `generateAreaMetadata` app/page.tsx:5.
  - `schema.ts` : Logic for creating JSON-LD scripts app/layout.tsx:4-9.
- **`lib/utils.ts`** : Contains the `cn()` utility, which merges Tailwind classes using `clsx` and `tailwind-merge` .

Sources: app/layout.tsx:4-9, app/page.tsx:5-24

Static Assets and Infrastructure

Public Directory (`public/`)

Stores static assets like the `site.webmanifest` app/layout.tsx:60. It also contains the favicon suite (e.g., `favicon-48x48.png`) generated by automated scripts public/favicon-48x48.png:1-17.

Scripts (`scripts/`)

The `scripts/generate-favicons.ts` utility is used to process a source SVG into the various PNG and ICO formats required for modern browsers and mobile devices CLAUDE.md:18.

CI/CD Configuration

The project includes a GitHub Actions workflow in `.github/workflows/ci.yml` that executes `npm run lint` and `npm run build` on every push to `dev` or `main` branches to ensure code quality .github/workflows/ci.yml:1-31. Dependency updates are managed by Renovate, with specific rules for Next.js and React packages renovate.json:1-25.

Sources: CLAUDE.md:11-19, .github/workflows/ci.yml:1-31, renovate.json:1-25

Application Pages & Routing

This section provides an overview of the **LegalOrbis** page hierarchy and routing architecture. The application utilizes the **Next.js App Router**, leveraging file-based routing to define the site structure. The routing strategy focuses on SEO-optimized dynamic paths for legal practice areas and a performant, component-driven home page.

Route Architecture

The application is structured into three primary route groups: the main landing page, dynamic legal area details, and error handling.

Visual Route Hierarchy

The following diagram maps the logical application structure to the physical file system.

Route-to-File Mapping

```
graph TD
    subgraph "Natural Language Space"
        Home["Home Page"]
        PracticeAreas["Practice Areas Detail"]
        Error404["404 Not Found"]
    end

    subgraph "Code Entity Space"
        RootPath["/"]
        SlugPath["/areas-juridicas/[slug]"]
        NotFoundPath["Any Invalid URL"]

        FileHome["app/page.tsx"]
        FileArea["app/areas-juridicas/[slug]/page.tsx"]
        FileNotFound["app/not-found.tsx"]
    end

    Home --> RootPath
    RootPath --> FileHome

    PracticeAreas --> SlugPath
    SlugPath --> FileArea

    Error404 --> NotFoundPath
    NotFoundPath --> FileNotFound
```

Sources: app/page.tsx:1-57, app/areas-juridicas/[slug]/page.tsx:1-127, app/not-found.tsx:1-50

2.1 Home Page

The Home Page acts as the central hub of the application. It is designed for high performance using **Next.js Dynamic Imports** to load heavy sections (like video and complex sliders) only when needed, reducing the initial bundle size.

- **Composition**: The page is a vertical stack of specialized sections including `HeroCarousel`, `QuienesSomos`, `VideoBanner`, and `AreasJuridicas`.
- **Performance**: Dynamic imports include loading skeletons (e.g., a black background for the `VideoBanner`) to maintain layout stability during hydration.
- **SEO**: Metadata is generated via `generatePageMetadata` to target broad keywords like "abogados Madrid".

For details, see Home Page.

Sources: app/page.tsx:7-24, app/page.tsx:26-42, app/page.tsx:47-54

2.2 Legal Area Detail Pages

The application uses dynamic routing to generate specific landing pages for different legal specialties (Penal, Civil, Laboral, etc.). These routes are defined under `app/areas-juridicas/[slug]`.

- **Slug Resolution**: The `slug` parameter is matched against the `areasData` object keys. If a match is not found, the `notFound()` function is invoked to trigger the 404 handler.
- **SEO & Schema**: Each page dynamically generates its own metadata and injects three types of JSON-LD structured data: `LegalService`, `BreadcrumbList`, and `FAQPage`.
- **Data Flow**: Component props are populated directly from the `areasData` object, ensuring UI consistency across different legal branches.

For details, see Legal Area Detail Pages.

Sources: app/areas-juridicas/[slug]/page.tsx:24-36, app/areas-juridicas/[slug]/page.tsx:52-58, app/areas-juridicas/[slug]/page.tsx:61-80, app/areas-juridicas/[slug]/page.tsx:105-122

2.3 404 Not Found Page

The custom `not-found.tsx` handler provides a branded experience for broken links or invalid slugs.

- **Recovery UX**: It provides explicit navigation buttons to return to the home page or browse the legal areas grid.
- **SEO Safety**: The page is explicitly marked with `robots: { index: false, follow: false }` to prevent search engines from indexing error states.
- **Visual Consistency**: It utilizes the brand's primary teal (`#1a5f5f`) and dark gradients to match the overall aesthetic.

For details, see 404 Not Found Page.

Sources: app/not-found.tsx:7-14, app/not-found.tsx:20-22, app/not-found.tsx:30-43

Page Navigation & Data Flow

The following diagram illustrates how data from the centralized `areasData` store flows into the dynamic routing system to render the practice area pages.

Data Flow Diagram

```
flowchart LR
    Data["areasData (lib/data)"] -- "Key Lookup" --> Resolver["Slug Resolver"]
    Resolver -- "Valid Slug" --> Page["AreaJuridicaDetail Component"]
    Resolver -- "Invalid Slug" --> Error["notFound() Function"]

    subgraph "Page Generation"
        Page --> Metadata["generateAreaMetadata()"]
        Page --> Schema["JSON-LD Schemas"]
        Page --> UI["Area Components"]
    end

    subgraph "Error Handling"
        Error --> Custom404["app/not-found.tsx"]
    end
```

Sources: app/areas-juridicas/[slug]/page.tsx:24-45, app/areas-juridicas/[slug]/page.tsx:52-58, app/areas-juridicas/[slug]/page.tsx:61-80

Home Page

The Home Page serves as the primary entry point for the Legal Orbis application. It is a single-page layout composed of several high-impact sections designed to introduce the firm's values, team, and legal expertise. The implementation leverages Next.js dynamic imports to optimize the initial bundle size and uses specialized animation components for a polished user experience.

Composition and Structure

The page is defined in `app/page.tsx` and follows a linear vertical composition. It integrates both static and dynamically imported components to balance performance and interactivity.

Component Layout

The `Home` component in app/page.tsx:44-56 renders the following sequence:

1. **Header**: Persistent navigation and contact triggers.
2. **HeroCarousel**: Above-the-fold visual impact with auto-advancing slides.
3. **QuienesSomos**: Detailed section about firm values and team composition.
4. **VideoBanner**: Immersive video background with brand messaging.
5. **AreasJuridicas**: Grid of practice areas linking to detail pages.
6. **ContactBanner**: Call-to-action for lead generation.
7. **Footer**: Office information and site-wide links.

Data Flow Diagram

The following diagram illustrates how the Home Page orchestrates its sub-components and where data/assets originate.

Home Page Component Architecture

```
graph TD
    subgraph "app/page.tsx"
        Page["Home Component"]
    end

    subgraph "Direct Imports"
        Header["Header Component"]
        Hero["HeroCarousel"]
    end

    subgraph "Dynamic Imports (Next/Dynamic)"
        QS["QuienesSomos"]
        VB["VideoBanner"]
        AJ["AreasJuridicas"]
        CB["ContactBanner"]
        FT["Footer"]
    end

    subgraph "External Assets & Data"
        Vids["/videos/banner.mp4"]
        Imgs["/images/jpg/*.jpg"]
        AD["areasData (lib/data)"]
    end

    Page --> Header
    Page --> Hero
    Page --> QS
    Page --> VB
    Page --> AJ
    Page --> CB
    Page --> FT

    Hero -.-> Imgs
    QS -.-> Imgs
    VB -.-> Vids
    AJ -.-> AD
```

Sources: app/page.tsx:1-57, components/hero-carousel.tsx:12-37, components/quienes-somos.tsx:12-34

Dynamic Loading Strategy

To improve the Core Web Vitals (specifically Largest Contentful Paint and Total Blocking Time), the Home Page utilizes `next/dynamic`. Components below the fold are loaded only when needed, with skeleton-style `loading` states to prevent layout shifts.

Component	Loading Strategy	Placeholder UI
`QuienesSomos`	Dynamic	Div with `.section-padding` and `#0B0B0B` background app/page.tsx:26-28
`VideoBanner`	Dynamic	Div with `h-screen` and black background app/page.tsx:30-32
`AreasJuridicas`	Dynamic	Div with `.section-padding` and `#0B0B0B` background app/page.tsx:34-36
`ContactBanner`	Dynamic	Div with `.section-padding` app/page.tsx:38-40
`Footer`	Dynamic	Default (null) app/page.tsx:42

Sources: app/page.tsx:26-42

Core Sections Implementation

HeroCarousel

The `HeroCarousel` components/hero-carousel.tsx:8-142 is a client-side component that manages an array of slides. - **State Management**: Uses `useState` for `currentSlide` and an `isMounted` flag to ensure client-side hydration before starting timers components/hero-carousel.tsx:9-10. - **Auto-advance**: A `setInterval` advances the slide every 6000ms components/hero-carousel.tsx:46-48. - **Animations**: Uses `CSSAnimatedSection` with the `fadeInUp` animation to reveal text content over the background images components/hero-carousel.tsx:91-101.

QuienesSomos (Firm Values)

The `QuienesSomos` component components/quienes-somos.tsx:9-144 handles the presentation of firm values (Compromiso, Profesionalidad, Transparencia). - **Interactive State**: Tracks `valorActivo` to switch between descriptions and images components/quienes-somos.tsx:10. - **Statistics Grid**: Displays a counter-style section with firm metrics (15+ Years, 500+ Cases) using `.border-t border-white/10` for visual separation components/quienes-somos.tsx:108-140.

VideoBanner

The `VideoBanner` components/video-banner.tsx:6-58 provides a cinematic break in the content. - **Video Implementation**: Uses a standard `<video>` tag with `autoPlay`, `muted`, and `loop` components/video-banner.tsx:10-23. - **Fallback**: Includes an `onError` handler that hides the video element if the source fails to load components/video-banner.tsx:17-19.

SEO and Metadata

The Home Page defines its SEO properties using the `generatePageMetadata` helper. This ensures the page has optimized tags for search engines and social media.

Metadata Configuration app/page.tsx:7-24: - **Title**: "Abogados en Madrid | Legal Orbis - Despacho Jurídico Especializado" - **Description**: Focuses on "multidisciplinar", "Madrid", and specific branches like "Penal, Civil, Laboral". - **Keywords**: A curated list including "abogados penalistas Madrid" and "asesoría legal Madrid". - **Canonical URL**: Set to `/`.

Code-to-SEO Mapping

```
graph LR
    subgraph "Code Entity Space"
        Metadata["metadata object (app/page.tsx)"]
        GenMeta["generatePageMetadata (lib/seo/metadata.ts)"]
    end

    subgraph "Natural Language Space (SEO)"
        Title["Browser Title Tag"]
```

```
        Desc["Meta Description"]
        Keyw["Meta Keywords"]
    end

    Metadata -- "calls" --> GenMeta
    GenMeta -- "produces" --> Title
    GenMeta -- "produces" --> Desc
    GenMeta -- "produces" --> Keyw
```

Sources: app/page.tsx:7-24, lib/seo/metadata.ts:1-20

Styling and Layout Utilities

The page relies on global utility classes defined in `app/globals.css` to maintain consistency across sections.

- `.section-padding`: Standardizes vertical spacing (`py-32`) and horizontal gutters app/globals.css:170-172.
- `.container-max`: Constrains the content width to `1600px` for ultra-wide displays app/globals.css:174-176.
- **CSS Animations**: Animations like `fadeInUp` and `fadeInLeft` are defined as CSS keyframes and applied via the `CSSAnimatedSection` component to avoid the overhead of large JS animation libraries for simple entrance effects app/globals.css:197-256.

Sources: app/globals.css:170-256, components/ui/css-animated-section.tsx:1-10

Legal Area Detail Pages

The Legal Area Detail pages are implemented via a dynamic Next.js route that serves as the deep-content layer for the firm's practice areas. These pages transform static data into SEO-optimized, interactive landing pages using a composition of specialized sub-components.

Slug Resolution and Data Flow

The route `app/areas-juridicas/[slug]/page.tsx` handles dynamic segments by looking up the `slug` against the `areasData` object app/areas-juridicas/[slug]/page.tsx:25-25.

Request Handling

1. **Slug Extraction**: The component receives `params` as a Promise, awaiting it to retrieve the `slug` app/areas-juridicas/[slug]/page.tsx:52-52.
2. **Data Lookup**: It attempts to find the corresponding entry in `areasData` app/areas-juridicas/[slug]/page.tsx:53-53.
3. **Error Handling**: If the slug does not match any key in the data model (e.g., `/areas-juridicas/invalid-area`), the system invokes the `notFound()` function app/areas-juridicas/[slug]/page.tsx:56-58, which triggers the custom 404 UI.

Data Flow Diagram

This diagram illustrates how the `slug` moves from the URL to the data layer and finally into the UI components.

Area Detail Data Resolution

```
graph TD
    URL["URL: /areas-juridicas/[slug]"] --> Page["AreaJuridicaDetail (app/areas-juridicas/[slug]
    Page --> Lookup["lib/data/areas-juridicas.ts: areasData[slug]"]

    subgraph DataValidation ["Data Validation"]
        Lookup --> Found{Area Found?}
        Found -- "No" --> NF["notFound()"]
        Found -- "Yes" --> Render["Render Page Components"]
    end

    Render --> Hero["AreaDetailHero"]
    Render --> Content["AreaContentSplit"]
    Render --> Why["AreaWhyChoose"]
    Render --> Branches["AreaDetailBranches"]
    Render --> FAQ["AreaFAQ"]
    Render --> Related["AreaRelatedLinks"]
```

Sources: app/areas-juridicas/[slug]/page.tsx:47-126, lib/data/areas-juridicas.ts:1-83

SEO and Structured Data

Each detail page performs heavy SEO lifting through two primary mechanisms: dynamic metadata generation and JSON-LD schema injection.

Metadata Generation

The `generateMetadata` function app/areas-juridicas/[slug]/page.tsx:19-45 uses the `generateAreaMetadata` factory to create unique titles, descriptions, and OpenGraph tags for every practice area. If an area is not found, it returns "noindex, nofollow" instructions to prevent crawlers from indexing invalid routes app/areas-juridicas/[slug]/page.tsx:27-36.

JSON-LD Injection

The page injects three types of structured data as `application/ld+json` scripts: * **LegalService**: Documents the specific legal services offered app/areas-juridicas/[slug]/page.tsx:61-65. * **BreadcrumbList**: Provides the navigation path (Home > Áreas Jurídicas > Current Area) app/areas-juridicas/[slug]/page.tsx:67-73. * **FAQPage**: Injected conditionally only if the area has defined FAQs app/areas-juridicas/[slug]/page.tsx:76-80.

SEO Entity Mapping

```
graph LR
    subgraph CodeEntitySpace ["Code Entity Space"]
        GAM["generateAreaMetadata()"]
        GLS["generateLegalServiceSchema()"]
        GBS["generateBreadcrumbSchema()"]
        GFS["generateFAQSchema()"]
    end

    subgraph SearchEngineSpace ["Search Engine / SEO Space"]
        Meta["HTML Head (Title, OG, Keywords)"]
        Service["Google Service Snippet"]
        Breadcrumb["Search Result Breadcrumbs"]
        FAQRich["Rich FAQ Results"]
    end

    GAM --> Meta
```

```
    GLS --> Service
    GBS --> Breadcrumb
    GFS --> FAQRich
```

Sources: app/areas-juridicas/[slug]/page.tsx:12-17, app/areas-juridicas/[slug]/page.tsx:60-103

Component Composition

The page is built from a suite of components that consume the `area` object.

AreaDetailHero

Displays the primary title, a decorative area number (e.g., "01"), and a high-level subtitle components/area-detail-hero.tsx:16-45. It uses a dark teal background (`#1A3635`) to establish the visual theme of the detail pages components/area-detail-hero.tsx:20-20.

AreaContentSplit & AreaWhyChoose

These components handle the `longDescription` field: * **AreaContentSplit**: Displays the first paragraph alongside a high-quality image components/area-content-split.tsx:21-65. * **AreaWhyChoose**: Displays the remaining paragraphs over a full-width background image with an overlay components/area-why-choose.tsx:23-44. It includes a `BottomSheet` trigger for the contact form components/area-why-choose.tsx:74-88.

AreaDetailBranches

This component renders the specific legal sub-disciplines. It has complex logic to handle different data formats: 1. **Simple List**: Renders a grid of cards with checkmarks components/area-detail-branches.tsx:141-170. 2. **Details Format**: If an item contains a colon (":"), it renders an accordion-style toggle where the text after the colon is hidden until expanded components/area-detail-branches.tsx:95-140. 3. **Laboral Exception**: Specifically forces a simple list layout for the Laboral area components/area-detail-branches.tsx:25-25.

AreaFAQ

Renders an accordion of frequently asked questions. It defaults to having the first FAQ expanded (`openIndex: 0`) components/area-faq.tsx:21-21. Like `AreaWhyChoose`, it provides a call-to-action button that opens the `ContactFormReusable` within a `BottomSheet` components/area-faq.tsx:82-96.

AreaRelatedLinks

Calculates "Other Areas" by filtering out the current area from the `areasData` object and taking the first three remaining entries components/area-related-links.tsx:23-25.

Sources: components/area-detail-hero.tsx, components/area-content-split.tsx, components/area-why-choose.tsx, components/area-detail-branches.tsx, components/area-faq.tsx, components/area-related-links.tsx

---

404 Not Found Page

This page documents the custom 404 error implementation in the LegalOrbis application. The custom 404 handler is designed to maintain brand consistency, prevent search engine indexing of broken links, and provide clear recovery paths for users who encounter non-existent routes.

Implementation Overview

The 404 page is implemented as a special Next.js file `app/not-found.tsx`. This component is automatically rendered by the Next.js App Router when a route does not match any defined pages or when the `notFound()` function is programmatically invoked from within a page or layout.

Metadata and SEO

To protect the site's SEO health, the page explicitly instructs crawlers not to index the error page or follow links from it.

Property	Value	Source
Title	Página no encontrada | Legal Orbis Abogados	app/not-found.tsx:8-8
Description	La página que buscas no existe.	app/not-found.tsx:9-9
Robots Index	`false`	app/not-found.tsx:11-11
Robots Follow	`false`	app/not-found.tsx:12-12

Sources: app/not-found.tsx:1-14

Component Structure

The `NotFound` component is a functional React component that utilizes the standard site layout elements (`Header` and `Footer`) to ensure the user does not feel "lost" or disconnected from the main application context.

Visual Composition

The page uses a full-height flex container to center the error message vertically between the header and footer.

Page Layout Diagram

```
graph TD
    subgraph "NotFound Page Structure [app/not-found.tsx]"
        Main["main (min-h-screen flex flex-col)"]
        H["Header Component"]
        Content["Content Div (flex-1 items-center justify-center)"]
        F["Footer Component"]
    end

    Main --> H
    Main --> Content
    Main --> F

    subgraph "Error Content [app/not-found.tsx:21-44]"
        Title["h1: 404"]
        Subtitle["h2: Página no encontrada"]
```

```
        Desc["p: Lo sentimos..."]
        Actions["Action Buttons Container"]
    end

    Content --> Title
    Content --> Subtitle
    Content --> Desc
    Content --> Actions
```

Sources: app/not-found.tsx:16-49

Brand Styling

The content area features a specific brand gradient and typography: * **Background**: A linear gradient from `--primary-dark` (`#1A3635`) to a near-black (`#0B0B0B`) app/not-found.tsx:20-20. * **Typography**: High-contrast white text for headings (`text-white`) and light gray for descriptions (`text-gray-300`) app/not-found.tsx:22-26.

Recovery Navigation

The 404 page provides two primary recovery paths to guide users back to valid content. Both paths utilize the custom `Button` primitive with the `asChild` pattern to wrap Next.js `Link` components.

Navigation Links

Target Route	Button Variant	Label	Implementation
`/`	Primary (`bg-[#1a5f5f]`)	Volver al inicio	app/not-found.tsx:30-35
`/areas-juridicas`	Outline (`border-white`)	Ver áreas jurídicas	app/not-found.tsx:36-42

Data Flow: Navigation Recovery

```
sequenceDiagram
    participant User
    participant NotFound as "app/not-found.tsx"
    participant Router as "Next.js Router"
```

```
User->>NotFound: Lands on invalid URL
Note over NotFound: Renders 404 UI
User->>NotFound: Clicks 'Volver al inicio'
NotFound->>Router: Push "/" [app/not-found.tsx:34]
User->>NotFound: Clicks 'Ver áreas jurídicas'
NotFound->>Router: Push "/areas-juridicas" [app/not-found.tsx:41]
```

Sources: app/not-found.tsx:29-43

Technical Dependencies

The 404 page integrates several core components and utilities to maintain consistency with the rest of the application:

- **Header**: app/not-found.tsx:3 - Provides the global navigation bar.
- **Footer**: app/not-found.tsx:4 - Provides office information and contact links.
- **Button (UI Primitive)**: app/not-found.tsx:5 - The shared button component used for the recovery links.
- **Metadata (Next.js)**: app/not-found.tsx:1 - Type definition for the SEO configuration object.

Sources: app/not-found.tsx:1-5

Component Library

The LegalOrbis component library is organized into a two-layer architecture designed for modularity and performance. The top layer consists of page-level section components that manage specific business logic and layout, while the bottom layer contains reusable UI primitives built on top of Radix UI and Tailwind CSS.

Component Architecture Overview

The system distinguishes between complex, domain-specific components located in `components/` and generic, atomic UI elements located in `components/ui/`.

Component Layering Model

```
graph TD
    subgraph "Page Layer"
        P1["app/page.tsx"]
        P2["app/areas-juridicas/[slug]/page.tsx"]
    end

    subgraph "Section Layer (components/)"
        S1["HeroCarousel"]
        S2["AreasJuridicas"]
        S3["AreaDetailHero"]
        S4["Header/Footer"]
    end

    subgraph "UI Primitive Layer (components/ui/)"
        U1["Button"]
        U2["BottomSheet"]
        U3["CSSAnimatedSection"]
        U4["Carousel"]
    end

    P1 --> S1
    P1 --> S2
    P2 --> S3
    S1 --> U3
    S2 --> U1
    S4 --> U2
    S3 --> U3
```

Sources: components/header.tsx:7-9, components/areas-juridicas.tsx:7-9, components/ui/bottom-sheet.tsx:3-6

Layout & Navigation Components

The `Header` and `Footer` components provide the global structural frame for the application. The `Header` is a sophisticated "scroll-aware" component that tracks the user's position to highlight active sections on the home page and adjusts its visual style (transparency vs. solid background) based on scroll depth. It also serves as the primary trigger for the contact modal, which utilizes a responsive `BottomSheet` pattern.

For details, see Layout & Navigation Components.

Sources: components/header.tsx:12-47, components/header.tsx:141-160, components/footer.tsx:35-95

Home Page Section Components

The home page is composed of several large-scale sections designed for high visual impact and engagement. Key components include the `HeroCarousel`, which features an auto-advancing image slider, and the `AreasJuridicas` component, which dynamically renders a grid of practice area cards based on the centralized data model.

For details, see Home Page Section Components.

Home Page Composition

```
graph TD
    subgraph "Home Page (app/page.tsx)"
        HC["HeroCarousel"]
        QS["QuienesSomos"]
        VB["VideoBanner"]
        AJ["AreasJuridicas"]
        CB["ContactBanner"]
    end

    AJ -->|Imports| AD["lib/data/areas-juridicas.ts"]
    HC -->|Uses| CAS["CSSAnimatedSection"]
    CB -->|Triggers| BSheet["BottomSheet"]
```

Sources: components/hero-carousel.tsx:12-37, components/areas-juridicas.tsx:15-102, components/contact-banner.tsx:14-89

Legal Area Detail Components

These components are specialized for the dynamic `[slug]` routes. They consume props derived from the `areasData` object to render practice-specific content,

including hero sections, service branches, and FAQs. They are optimized for SEO and content readability.

For details, see Legal Area Detail Components.

Sources: components/areas-juridicas.tsx:21-23

Contact Form Components

The application features two primary contact form implementations: a full-page version and a reusable version (`ContactFormReusable`) designed for use within modals and bottom sheets. Both share a common validation schema and integrate with Formspree for submission handling.

For details, see Contact Form Components.

Sources: components/contact-banner.tsx:7-10, components/header.tsx:9-15

UI Primitives

Located in `components/ui/` , these are the building blocks of the entire interface. They are designed to be accessible and performant, utilizing `IntersectionObserver` for animations and Radix UI for complex interactions like dialogs and accordions.

For details, see UI Primitives.

UI Component Mapping | UI Component | Code Entity | Primary Responsibility | | :--- | :--- | :--- | | **Animation Wrapper** | `CSSAnimatedSection` | Intersection-based CSS transitions | | **Responsive Modal** | `BottomSheet` | Dialog on desktop, drawer on mobile | | **Value Display** | `ValorDialog` | Interactive display for firm values | | **Action Element** | `Button` | Styled buttons using CVA variants |

Sources: components/ui/css-animated-section.tsx:12-56, components/ui/bottom-sheet.tsx:100-175, components/ui/valor-dialog.tsx:17-50

Layout & Navigation Components

The layout and navigation system in LegalOrbis provides a consistent user experience across the home page and practice area details. It manages scroll-dependent styling, active section tracking for single-page navigation, and a unified contact modal pattern that adapts to mobile and desktop viewports.

Header Component

The `Header` component serves as the primary navigation hub. It implements a "sticky" behavior that transitions between a transparent state (at the top of the page) and a solid, shadowed state upon scrolling.

Scroll and Section Tracking

The component uses a `useEffect` hook to monitor the `window.scrollY` position components/header.tsx:19-51. - **Scroll Awareness**: If the scroll position exceeds 100px, `isScrolled` is set to true, triggering a CSS transition from `bg-black/20` to `bg-white/95` components/header.tsx:70-74. - **Active Section Tracking**: On the home page, the header calculates which section is currently in view by comparing the scroll position against the `offsetTop` and `offsetHeight` of specific DOM elements (`hero` , `quienes-somos` , `areas-juridicas` , `contacto`) components/header.tsx:24-45.

Navigation Logic

- **Smooth Scrolling**: The `scrollToSection` function uses `element.scrollIntoView({ behavior: 'smooth' })` to navigate to home page anchors components/header.tsx:53-61.
- **Dynamic Assets**: The brand logo toggles between `LegalOrbisWhite.png` and `LegalOrbis.png` based on the `isScrolled` state components/header.tsx:84-95.

Mobile Navigation

A hamburger menu is implemented for mobile devices. It toggles the `isMobileMenuOpen` state, which renders a full-screen overlay containing the navigation links components/header.tsx:163-200.

Sources: - `components/header.tsx` 11-200

Footer Component

The `Footer` component provides organizational information, office locations, and quick access to site sections.

Structure and Content

- **Office Info**: Displays the Madrid office address and contact details components/footer.tsx:41-56.
- **Quick Links**: Maps through a `quickLinks` array to provide navigation to internal sections using the `scrollToSection` helper components/footer.tsx:13-18.
- **Copyright**: Automatically updates the year using `new Date().getFullYear()` components/footer.tsx:102-103.

Interaction

The footer includes a direct link to the contact section (`#contacto`), ensuring users have a clear path to conversion at the end of every page components/footer.tsx:84-91.

Sources: - `components/footer.tsx` 5-115

Contact Modal Pattern

LegalOrbis uses a unified pattern for the contact form, leveraging the `BottomSheet` component. This pattern is triggered from the `Header`, the `ContactBanner`, and various CTA buttons.

Implementation: BottomSheet & Dialog

The `BottomSheet` component is a wrapper around Radix UI's `DialogPrimitive` components/ui/bottom-sheet.tsx:4-32. It dynamically changes its behavior based on the device: - **Desktop**: Renders as a centered modal (`Dialog`) components/ui/bottom-sheet.tsx:134-150. - **Mobile**: Renders as a drawer that slides up from the bottom components/ui/bottom-sheet.tsx:100-131.

Component Relationship Diagram

This diagram shows how the `Header` and `ContactBanner` interact with the `BottomSheet` primitive to render the `ContactForm`.

```
graph TD
    subgraph "Navigation Components"
        H["Header (components/header.tsx)"]
        CB["ContactBanner (components/contact-banner.tsx)"]
    end

    subgraph "UI Primitives"
        BS["BottomSheet (components/ui/bottom-sheet.tsx)"]
        BSC["BottomSheetContent"]
    end

    subgraph "Business Logic"
        CF["ContactForm (components/contact-form-reusable.tsx)"]
    end

    H -- "triggers" --> BS
    CB -- "triggers" --> BS
    BS -- "contains" --> BSC
    BSC -- "renders" --> CF
```

Sources: - `components/header.tsx` 141-160 - `components/contact-banner.tsx` 70-82 - `components/ui/bottom-sheet.tsx` 28-175

Data Flow: Modal Interaction

The following diagram illustrates the state management for opening the contact modal and the touch-drag logic for mobile users.

```
sequenceDiagram
    participant User
    participant Header as "Header Component"
    participant BS as "BottomSheet (Radix Dialog)"
    participant Hook as "useIsMobile Hook"

    User->>Header: Click "CONTACTAR"
    Header->>Header: setIsContactModalOpen(true)
    Header->>BS: Render <BottomSheet open={true}>
    Hook->>BS: Return isMobile (true/false)

    alt isMobile == true
        BS->>User: Show Slide-up Drawer
        User->>BS: Touch Start/Move (Drag Down)
        BS->>BS: handleTouchMove (Update translateY)
        Note over BS: if deltaY > 100px
        BS->>Header: Trigger onOpenChange(false)
    else isMobile == false
        BS->>User: Show Centered Dialog
    end
```

Sources: - `components/ui/bottom-sheet.tsx` 12-26 (useIsMobile) - `components/ui/bottom-sheet.tsx` 65-98 (Touch handlers) - `components/header.tsx` 15-16 (State)

Reusable UI Integration

The layout components rely on several `components/ui/` primitives to maintain visual consistency:

Component	Usage in Layout	File Reference
`Button`	Navigation links and CTA triggers	components/ui/button.tsx
`BottomSheet`		

Component	Usage in Layout	File Reference
	Wraps the contact form in Header and Banner	components/ui/bottom-sheet.tsx:28
`CSSAnimatedSection`	Provides entrance animations for Banner content	components/ui/css-animated-section.tsx
`ValorDialog`	Reuses the BottomSheet pattern for "Firm Values"	components/ui/valor-dialog.tsx:17

Sources: - `components/contact-banner.tsx` 27-43 - `components/ui/valor-dialog.tsx` 1-51

Home Page Section Components

This page documents the high-level section components that compose the LegalOrbis home page. These components are designed for high visual impact, utilizing a combination of **Tailwind CSS**, **Framer Motion** (via `CSSAnimatedSection`), and **Next.js Image/Video** optimization.

Overview of Sections

The home page is built as a sequence of full-width sections, most of which utilize the `section-padding` and `container-max` utility classes defined in `app/globals.css` 170-176.

Component Hierarchy & Data Flow

The following diagram illustrates how the home page sections relate to the underlying data structures and UI primitives.

Home Page Component Architecture

```
graph TD
    subgraph "Page Entity Space"
```

```
        P["app/page.tsx"]
    end

    subgraph "Section Components"
        HC["HeroCarousel"]
        QS["QuienesSomos"]
        VB["VideoBanner"]
        AJ["AreasJuridicas"]
    end

    subgraph "Data & Primitives"
        AD["areasData (lib/data/areas-juridicas.ts)"]
        CAS["CSSAnimatedSection"]
        VD["ValorDialog"]
    end

    P --> HC
    P --> QS
    P --> VB
    P --> AJ

    AJ -- "Iterates over" --> AD
    QS -- "State: valorActivo" --> VD
    HC -- "Wraps content" --> CAS
    QS -- "Wraps content" --> CAS
    AJ -- "Wraps content" --> CAS
```

Sources: components/hero-carousel.tsx:8-142, components/quienes-somos.tsx:9-144, components/areas-juridicas.tsx:11-107, lib/data/areas-juridicas.ts:1-200

HeroCarousel

The `HeroCarousel` serves as the primary entry point of the site. It is a full-screen (`h-screen`) auto-advancing slider.

Implementation Details

- **State Management**: Uses `currentSlide` to track the active index 9.
- **Auto-Advance**: A `useEffect` hook sets an interval of 6000ms to advance the slide 46-51. It includes a safety check for `isMounted` to prevent hydration mismatches 40-44.

- **Transitions**: Slides use absolute positioning and toggle `opacity-100` vs `opacity-0` with a `duration-1000` transition 71-73.
- **Optimization**: The first image in the array uses the `priority` prop for LCP (Largest Contentful Paint) optimization 80.

Sources: components/hero-carousel.tsx:8-142

QuienesSomos (About Us)

This section details the firm's values and multidisciplinary approach. It features an interactive tab-like navigation system for different organizational values.

Value Navigation Logic

The component maintains a `valorActivo` state 10. When a user clicks a dot navigation button, the state updates, triggering a content swap of the title, description, and image 75-87.

Feature	Implementation
State	`valorActivo: number` 10
Data Structure	Array of objects containing `palabra`, `titulo`, `descripcion`, and `imagen` 12-34
Statistics Grid	A four-column grid displaying firm milestones (e.g., 15+ years experience) 108-140

Sources: components/quienes-somos.tsx:9-144

AreasJuridicas

This component renders a grid of the firm's practice areas. It dynamically maps over the `areasData` object imported from the library.

Data Interaction

- **Dynamic Mapping**: It converts the `areasData` object into an array using `Object.values(areasData)` 15.
- **Navigation**: Uses the Next.js `useRouter` hook to navigate to specific detail pages (e.g., `/areas-juridicas/penal`) when the "CONOCER MÁS" button is clicked 21-23, 91-96.
- **Visuals**: Each card features a background image with a dark overlay (`bg-black/60`) to ensure text readability 70.

Practice Area Rendering Flow

```
sequenceDiagram
    participant AJ as AreasJuridicas.tsx
    participant AD as lib/data/areas-juridicas.ts
    participant R as next/navigation (Router)

    AJ->>AD: Import areasData
    AD-->>AJ: Return AreaData object
    AJ->>AJ: Object.values(areasData).map()
    Note over AJ: Render Cards with Framer Motion
    AJ->>R: router.push(/areas-juridicas/[id])
```

Sources: components/areas-juridicas.tsx:11-107, lib/data/areas-juridicas.ts:9-10

VideoBanner

A high-impact visual section featuring a background video loop.

Technical Configuration

- **Video Attributes**: The `<video>` element is configured with `autoPlay`, `muted`, `loop`, and `playsInline` for mobile compatibility 10-14.
- **Performance**: Uses `preload="none"` to save bandwidth until the section is near the viewport 15.
- **Error Handling**: Includes an `onError` handler that hides the video element if the source fails to load, preventing a broken UI 17-19.

- **Overlay**: A `bg-black/60` div is layered over the video to maintain a minimum contrast ratio for the white text 24.

Sources: components/video-banner.tsx:6-61

Animation Strategy

All sections utilize `CSSAnimatedSection` to handle entry animations. This component abstracts the intersection observer logic and applies CSS classes defined in `app/globals.css`.

Available Animations

The following animations are defined in the global CSS layer 197-256: - `fadeInUp`: Translates from `30px` to `0` while fading in. - `fadeInLeft`: Translates from `-30px` to `0`. - `fadeInRight`: Translates from `30px` to `0`. - `fadeInScale`: Scales from `0.95` to `1`.

Sources: app/globals.css:197-256, components/ui/css-animated-section.tsx:1-50

Legal Area Detail Components

The Legal Area Detail components are a specialized suite of React components designed to render the practice area pages (e.g., `/areas-juridicas/penal`). These components are optimized for high-readability legal content, featuring sophisticated typography, CSS-driven animations, and a data-driven architecture that consumes the `areasData` model.

Component Overview and Data Flow

The data flow for these components starts at the dynamic route `app/areas-juridicas/[slug]/page.tsx`, which fetches the relevant `AreaData` object from `lib/`

`data/areas-juridicas.ts`. This object is then passed down as props to the various detail components.

Data Flow Diagram

The following diagram illustrates how the central data store populates the specific UI components on a practice area page.

Data Propagation to Detail Components

```
graph TD
    subgraph "Data Layer"
        DATA["areasData (lib/data/areas-juridicas.ts)"]
    end

    subgraph "Page Layer"
        PAGE["AreaDetailPage (app/areas-juridicas/[slug]/page.tsx)"]
    end

    subgraph "Detail Components"
        HERO["AreaDetailHero"]
        SPLIT["AreaContentSplit"]
        WHY["AreaWhyChoose"]
        BRANCH["AreaDetailBranches"]
        FAQ["AreaFAQ"]
        REL["AreaRelatedLinks"]
    end

    DATA -- "Area Object" --> PAGE
    PAGE -- "id, title, subtitle" --> HERO
    PAGE -- "longDescription, image" --> SPLIT
    PAGE -- "longDescription, image" --> WHY
    PAGE -- "id, branches, services" --> BRANCH
    PAGE -- "faqs" --> FAQ
    PAGE -- "relatedAreas" --> REL
```

Sources: lib/data/areas-juridicas.ts:1-10, components/area-detail-hero.tsx:6-15, components/area-detail-branches.tsx:8-15

Component Technical Details

AreaDetailHero

The entry point of the detail page. It displays the area title, a descriptive subtitle, and a large decorative background number corresponding to the practice area's index.

- **Implementation:** Uses `CSSAnimatedSection` with `fadeInLeft` and `fadeInUp` variants components/area-detail-hero.tsx:25-40.
- **Visual Elements:** Features a large background digit rendered with `text-white/10` and a `blur-3xl` gradient overlay components/area-detail-hero.tsx:50-70.

AreaContentSplit & AreaDetailContent

These components handle the primary textual description of the legal area.

- **AreaContentSplit:** Typically used for the introduction. It extracts the first paragraph from `longDescription` (by splitting on `\n\n`) and pairs it with an optimized `next/image` components/area-content-split.tsx:16-21.
- **AreaDetailContent:** A more complex layout used for longer narratives. It features a desktop-specific "zigzag" layout where text is split across the top and bottom of a central image components/area-detail-content.tsx:70-128.

AreaDetailBranches

This component manages the display of specific legal sub-specialties. It is highly polymorphic and adjusts its rendering logic based on the data format.

- **Simple List:** If the data is a flat array of strings, it renders a grid of checkmark items components/area-detail-branches.tsx:143-170.
- **Accordion Format:** If items contain a colon (`:`), it parses them into title/detail pairs and renders an interactive accordion components/area-detail-branches.tsx:95-139.
- **Complex Branches:** If the data contains `Branch` objects, it renders detailed cards with descriptions components/area-detail-branches.tsx:173-180.

Branch Parsing Logic

```
flowchart TD
    START["Receive area.services or area.branches"] --> TYPE_CHECK{"Format?"}
    TYPE_CHECK -- "String with ':'" --> PARSE["parseItem() splits by ':'"]
    PARSE --> ACCORDION["Render Accordion UI"]
    TYPE_CHECK -- "Flat String" --> GRID["Render Checkmark Grid"]
    TYPE_CHECK -- "Branch Object" --> CARDS["Render Detailed Cards"]
```

Sources: components/area-detail-branches.tsx:20-50, components/area-detail-branches.tsx:95-170

AreaWhyChoose

Focuses on the firm's value proposition for that specific area. * **Data Handling:** It skips the first paragraph (used by `AreaContentSplit`) and renders the remaining paragraphs from `longDescription` components/area-why-choose.tsx:23-28. * **Interactivity:** Includes a CTA button that triggers a `BottomSheet` containing a `ContactForm` components/area-why-choose.tsx:74-88.

AreaFAQ

Renders a practice-area-specific FAQ section. * **State Management:** Uses a local `openIndex` state to manage accordion toggling components/area-faq.tsx:21-26. * **SEO:** While this component handles the UI, the corresponding JSON-LD structured data is injected at the page level using `generateFAQSchema` lib/seo/schema.ts:1-10.

AreaRelatedLinks

Provides cross-linking between different practice areas to improve SEO and user retention. * **Filtering:** Automatically filters out the `currentAreaId` and limits the display to 3 related areas components/area-related-links.tsx:23-25. * **Styling:** Uses a dark teal background (`#1A3635`) with white text and hover-transformed arrows components/area-related-links.tsx:32-80.

Shared Technical Patterns

Animation System

All components in this suite utilize `CSSAnimatedSection` for entry animations. This component applies Tailwind-based CSS animations (from `tw-animate-css`) when the element enters the viewport.

Component	Common Animation	Trigger Delay
Hero	`fadeInLeft` / `fadeInUp`	0.1s - 0.3s
Content	`fadeInScale`	0.4s - 0.5s
Branches	`fadeInUp` / `fadeInScale`	Staggered (index * 0.05s)

Sources: components/area-detail-branches.tsx:107, components/area-detail-hero.tsx:25, components/area-content-split.tsx:39

Theming and Visuals

The detail pages use a distinct color palette compared to the home page: * **Primary Background:** Deep black (`#0B0B0B`) components/area-detail-content.tsx:18 or Dark Teal (`#1A3635`) components/area-detail-hero.tsx:20. * **Decorative Elements:** Use `animate-pulse` on absolute-positioned divs with high blur (`blur-3xl`) to create a "glowing" effect components/area-detail-branches.tsx:56-65.

Sources: components/area-detail-branches.tsx:52-65, components/area-detail-hero.tsx:77-80, components/area-content-split.tsx:28-34

Contact Form Components

This page documents the contact form implementations within the LegalOrbis codebase. The system provides two primary interfaces for user inquiries: a comprehensive full-page section and a flexible, reusable component designed for

modal integration. Both implementations utilize a custom React hook to manage state and interface with the Formspree API.

Core Logic: useContactForm Hook

The `useContactForm` hook centralizes the state management and submission logic for all contact forms. It abstracts the complexities of handling input changes, managing submission states (loading, success, error), and communicating with the backend service.

Implementation Details

- **Data Schema**: Defined by the `ContactFormData` interface, capturing `nombre`, `telefono`, `email`, `asunto`, and `mensaje` hooks/useContactForm.ts:3-9.
- **State Management**: Uses `useState` to track form values and a boolean `isSubmitting` to prevent duplicate submissions hooks/useContactForm.ts:30-31.
- **Integration**: Sends a `POST` request to the Formspree endpoint `https://formspree.io/f/manlbdwz` with a JSON payload hooks/useContactForm.ts:55-61.

Data Flow Diagram

This diagram illustrates how the `useContactForm` hook mediates between the UI components and the external Formspree API.

Title: Contact Form Submission Data Flow

```
graph TD
    subgraph "Code Entity Space"
        A["ContactFormReusable (UI)"] -- "calls" --> B["handleSubmit()"]
        C["ContactForm (UI)"] -- "calls" --> B
        B -- "contained in" --> D["useContactForm Hook"]
        D -- "POST JSON" --> E["Formspree API"]
    end

    subgraph "Natural Language Space"
        E -- "Response 200 OK" --> F["Success Alert / Callback"]
        E -- "Response Error" --> G["Error Alert"]
    end
```

Sources: hooks/useContactForm.ts:29-91, components/contact-form-reusable.tsx:23-24

ContactForm (Full Page)

The `ContactForm` component in `components/contact-form.tsx` is a full-width section typically used on the home page. It includes brand-specific contact information (phone, email, address) alongside the input fields components/contact-form.tsx:82-133.

Key Features

- **Contact Info Grid**: Displays office details using `lucide-react` icons like `Phone`, `Mail`, `MapPin`, and `Clock` components/contact-form.tsx:54-79.
- **Interactive Map Placeholder**: Includes a UI block for an interactive map components/contact-form.tsx:124-132.
- **Local State**: Unlike the reusable version, this implementation currently maintains its own internal state and a simulated 2-second delay for submission components/contact-form.tsx:7-17, components/contact-form.tsx:37.

Sources: components/contact-form.tsx:1-153

ContactFormReusable

The `ContactFormReusable` component is a streamlined version of the form designed for flexibility. It is primarily used within the `Header` contact modal (via `BottomSheet` or `Dialog`).

Configuration Props

The component accepts several props to modify its appearance and behavior: | Prop | Type | Description | | :--- | :--- | :--- | | `onSuccess` | `() => void` | Callback executed after successful Formspree submission (e.g., to close a modal). | | `showTitle` | `boolean` | Controls visibility of the header text (default: `true`). | | `title` | `string` |

Custom heading text. | | `className` | `string` | Additional CSS classes for the container. |

Component Architecture

This diagram maps the UI elements to the underlying state management in `useContactForm`.

Title: ContactFormReusable Component Structure

```
graph LR
    subgraph "UI Elements (components/contact-form-reusable.tsx)"
        Input1["Input (nombre)"]
        Input2["Input (telefono)"]
        Input3["Input (email)"]
        TextArea["Textarea (mensaje)"]
        SubmitBtn["Button (submit)"]
    end

    subgraph "Logic (hooks/useContactForm.ts)"
        State["formData State"]
        ChangeHandler["handleInputChange"]
        SubmitHandler["handleSubmit"]
    end

    Input1 & Input2 & Input3 & TextArea -- "onChange" --> ChangeHandler
    ChangeHandler -- "updates" --> State
    SubmitBtn -- "onClick" --> SubmitHandler
    SubmitHandler -- "reads" --> State
```

Sources: components/contact-form-reusable.tsx:8-22, components/contact-form-reusable.tsx:37-137, hooks/useContactForm.ts:33-41

Form Field Schema

The following table details the fields required by the `ContactFormData` interface and their corresponding HTML attributes in the components.

Field Name	Type	Required	UI Component	File Reference
`nombre`	`string`	Yes	`Input` (text)	hooks/useContactForm.ts:4
`telefono`	`string`	Yes	`Input` (tel)	hooks/useContactForm.ts:5
`email`	`string`	Yes	`Input` (email)	hooks/useContactForm.ts:6
`asunto`	`string`	No	`Input` (text)	hooks/useContactForm.ts:7
`mensaje`	`string`	No	`Textarea`	hooks/useContactForm.ts:8

Sources: hooks/useContactForm.ts:3-9, components/contact-form-reusable.tsx:38-125

UI Primitives

The `components/ui/` layer contains the foundational building blocks of the LegalOrbis interface. These components are designed to be highly reusable, following the **Atomic Design** philosophy. The system leverages **Radix UI** for accessible primitives, **Framer Motion** and **Intersection Observer** for animations, and **Tailwind CSS** with the **CVA (Class Variance Authority)** pattern for styling.

Component Pattern & Utilities

The UI library relies on a centralized utility for class merging and conditional styling.

`cn()` Utility

Located in `lib/utils.ts`, the `cn()` function combines `clsx` and `tailwind-merge`. This allows for conditional class application while ensuring that Tailwind CSS utility classes are merged correctly without conflicts (e.g., preventing duplicate padding classes) lib/utils.ts:5-7.

CVA (Class Variance Authority)

Components like `Button` (implied by standard Shadcn patterns in this stack) use CVA to define variants (e.g., `primary`, `outline`) and sizes in a type-safe manner.

Animation Primitives

LegalOrbis provides three distinct strategies for scroll-triggered animations to balance performance and complexity.

AnimatedSection Components Comparison

Component	Engine	Trigger	Best For
`AnimatedSection`	Framer Motion	`whileInView`	Complex sequences, spring physics
`LazyAnimatedSection`	Framer Motion	`viewport`	Optimized Framer Motion usage components/ui/lazy-animated-section.tsx:54-54
`CSSAnimatedSection`	Vanilla JS + CSS	`IntersectionObserver`	Low-overhead, simple fade/slide effects components/ui/css-animated-section.tsx:22-33

Implementation Detail: LazyAnimatedSection

Uses `motion.div` from Framer Motion with predefined variants for `fadeInUp`, `fadeInLeft`, `fadeInScale`, etc. components/ui/lazy-animated-section.tsx:6-31. It triggers when 20% of the element is visible with a -50px margin components/ui/lazy-animated-section.tsx:54-54.

Implementation Detail: CSSAnimatedSection

Uses a manual `IntersectionObserver` to toggle a boolean `isVisible` state components/ui/css-animated-section.tsx:19-25. Once visible, it applies CSS classes (e.g., `animate-fadeInUp`) defined in the global Tailwind configuration components/ui/css-animated-section.tsx:45-47.

Sources: components/ui/lazy-animated-section.tsx:1-60, components/ui/css-animated-section.tsx:1-56

Overlays & Dialogs

LegalOrbis implements a hybrid overlay system that adapts to the user's device.

BottomSheet

A polymorphic component built on `@radix-ui/react-dialog`. It provides a standard modal dialog on desktop and a swipeable drawer on mobile components/ui/bottom-sheet.tsx:100-132.

Key Logic: - `useIsMobile`: Tracks window width to toggle between `fixed center` (Desktop) and `fixed bottom` (Mobile) layouts components/ui/bottom-sheet.tsx:12-26. - **Touch Handling**: Implements `handleTouchStart`, `handleTouchMove`, and `handleTouchEnd` to allow users to swipe the drawer down to close it on mobile devices components/ui/bottom-sheet.tsx:65-98. - **Desktop Close**: A floating close button positioned at the top-right of the viewport for desktop users components/ui/bottom-sheet.tsx:152-172.

ValorDialog

A specialized wrapper for the `BottomSheet` used in the "Quienes Somos" section. It accepts a `valor` object and renders a styled layout containing an image, title, and description components/ui/valor-dialog.tsx:17-50.

Entity Mapping: Overlay System

```
graph TD
    subgraph "Natural Language Space"
        Modal["User Modal/Drawer"]
        Trigger["Clickable Element"]
        Values["Firm Values Content"]
    end

    subgraph "Code Entity Space"
        BS["BottomSheet (components/ui/bottom-sheet.tsx)"]
        BSC["BottomSheetContent"]
        VD["ValorDialog (components/ui/valor-dialog.tsx)"]
        RP["@radix-ui/react-dialog"]
    end

    Modal --> BS
    BS --> RP
    Trigger --> VD
    Values --> VD
    VD --> BSC
```

Sources: components/ui/bottom-sheet.tsx:1-178, components/ui/valor-dialog.tsx:1-51

Data-Driven UI Components

Accordion & Carousel

- **Accordion**: Built using Radix UI primitives. Used primarily in practice area detail pages for FAQs.
- **Carousel**: Built using the `embla-carousel-react` library. It powers the `HeroCarousel` and allows for smooth, touch-enabled sliding with autoplay capabilities.

Form Inputs

Standardized `Input` and `Textarea` components are used within the `ContactForm`. They are styled with a consistent teal focus ring (`--primary`) and gray borders to match the brand identity.

UI Component Architecture

The following diagram illustrates how the UI primitives interact with the library layer and the styling system.

Component Data Flow & Styling

```
graph LR
    subgraph "Styling Layer"
        TW["Tailwind CSS 4"]
        CN["cn() Utility"]
        CVA["Class Variance Authority"]
    end

    subgraph "UI Primitives (components/ui/)"
        Button["Button"]
        Input["Input"]
        BS["BottomSheet"]
        AS["AnimatedSection"]
    end

    subgraph "External Dependencies"
        Radix["@radix-ui/react-*"]
        Framer["framer-motion"]
        Embla["embla-carousel-react"]
    end

    Button --> CN
    Button --> CVA
    BS --> Radix
    AS --> Framer
    CN --> TW
```

Sources: lib/utils.ts:1-7, components/ui/bottom-sheet.tsx:4-6, components/ui/lazy-animated-section.tsx:3-3

Usage Examples

Using CSSAnimatedSection

```
// components/ui/css-animated-section.tsx
<CSSAnimatedSection animation="fadeInLeft" delay={0.2}>
  <p>This content slides in from the left using CSS animations.</p>
</CSSAnimatedSection>
```

Implementing a BottomSheet Trigger

```
// components/ui/bottom-sheet.tsx
<BottomSheet>
  <DialogPrimitive.Trigger>Open Modal</DialogPrimitive.Trigger>
  <BottomSheetContent>
    <h2>Content Title</h2>
    <p>This will be a drawer on mobile and a modal on desktop.</p>
  </BottomSheetContent>
</BottomSheet>
```

Sources: components/ui/css-animated-section.tsx:12-17, components/ui/bottom-sheet.tsx:28-32, components/ui/bottom-sheet.tsx:51-58

Data & Content Layer

The **Data & Content Layer** serves as the central repository for the application's domain knowledge and functional logic. It manages the structured data representing the firm's legal expertise and provides the core utility functions used to maintain styling consistency across the React component tree.

By decoupling the legal content from the UI components, the system ensures that updates to practice areas, FAQs, or SEO keywords can be managed in a single location without modifying page structures.

Core Architecture

The layer is organized into two primary concerns within the `lib/` directory:

1. **Domain Data**: Static definitions of legal services, branching logic, and practice area metadata.
2. **Shared Utilities**: Helper functions for DOM manipulation, class merging, and path resolution.

System Mapping: Natural Language to Code Entities

The following diagram illustrates how conceptual legal entities are mapped to specific TypeScript structures and utility functions within the codebase.

Content & Utility Mapping

```
graph TD
    subgraph "Natural Language Space"
        A["Practice Area (e.g., Penal)"]
        B["Legal Specialization"]
        C["Common Questions"]
        D["Dynamic Styling"]
    end

    subgraph "Code Entity Space (lib/)"
        A --> E["areasData (Object)"]
        B --> F["AreaData.branches (Array)"]
        C --> G["AreaData.faqs (Array)"]
        D --> H["cn() (Function)"]
    end

    subgraph "Files"
        E & F & G --- I["lib/data/areas-juridicas.ts"]
        H --- J["lib/utils.ts"]
    end
```

Sources: lib/data/areas-juridicas.ts:1-200, lib/utils.ts:1-7

Legal Areas Data Model

The application's content is driven by a centralized data model located in `lib/data/areas-juridicas.ts`. This file defines the five core practice areas: **Penal, Civil, Laboral, Mercantil, and Administrativo**.

Each area is represented by a structured object that contains: * **Identification**: Unique IDs and slugs used for dynamic routing. * **Marketing Copy**: Subtitles, descriptions, and long-form content for detail pages. * **Service Hierarchy**: Specific branches of law and bulleted service lists. * **SEO Metadata**: Localized meta titles, descriptions, and keyword arrays used by the SEO subsystem. * **FAQ Sets**: Question-and-answer pairs specific to that legal domain.

For a detailed breakdown of the TypeScript interfaces and the full data structure, see Legal Areas Data Model.

Sources: lib/data/areas-juridicas.ts:2-44

Utility Functions

The `lib/utils.ts` file provides the foundational logic for the application's presentation layer. Its primary export is the `cn()` function, which facilitates conditional styling and Tailwind CSS class merging.

The `cn()` Helper

The `cn()` function wraps `clsx` and `tailwind-merge` to solve two common issues in React development: 1. **Conditional Logic**: Easily toggling classes based on component state. 2. **Tailwind Conflicts**: Ensuring that classes passed via props correctly override default classes without specificity issues.

Path Aliasing

The project utilizes a TypeScript path alias `@/*` (configured in `tsconfig.json`) which points to the root directory. This allows for clean, non-relative imports across the `lib/`, `components/`, and `app/` directories.

For technical details on implementation and usage patterns, see Utility Functions.

Sources: lib/utils.ts:1-7

Data Flow Diagram

The following diagram shows how data from the `lib/` directory flows into the UI and SEO layers.

Data Flow Architecture

```
flowchart LR
    subgraph "Data Layer (lib/data/)"
        DATA["areasData"]
    end

    subgraph "Processing (lib/)"
        UTIL["utils.ts (cn)"]
        SEO["seo/ (metadata)"]
    end

    subgraph "UI Layer (app/ & components/)"
        PAGE["Dynamic Area Page"]
        COMP["UI Components"]
    end

    DATA --> SEO
    DATA --> PAGE
    UTIL --> COMP
    SEO --> PAGE
    COMP --> PAGE
```

Sources: lib/data/areas-juridicas.ts:1-10, lib/utils.ts:1-7

Legal Areas Data Model

The Legal Orbis platform relies on a centralized data structure to manage its legal practice areas. This model ensures consistency across the user interface, search engine optimization (SEO), and automated site indexing. By decoupling content from presentation, the system can dynamically generate detail pages, metadata, and structured data schemas from a single source of truth.

Core Data Structures

The data model is defined in `lib/data/areas-juridicas.ts` using TypeScript interfaces to enforce strict typing across the application. The model is built around the `areasData` constant, which serves as the primary registry for all legal specialties.

Type Definitions

The system utilizes two primary interfaces to describe legal content:

Interface	Purpose
`Branch`	A simple string representing a specific legal sub-specialty or service line.
`AreaData`	A comprehensive object containing marketing copy, SEO tags, service lists, and FAQ items for a specific area.

The `areasData` Registry

The `areasData` object is a dictionary where keys are URL slugs (e.g., `penal`, `civil`) and values are `AreaData` objects. The firm currently supports five primary areas:

1. **Penal**: Criminal and Penitentiary Law lib/data/areas-juridicas.ts:3-79.
2. **Civil**: General Civil Law, Contracts, and Torts lib/data/areas-juridicas.ts:80-165.
3. **Laboral**: Employment and Social Security Law lib/data/areas-juridicas.ts:166-240.

4. **Mercantil**: Corporate and Commercial Law lib/data/areas-juridicas.ts:241-314.
5. **Administrativo**: Public Law and Administrative Procedures lib/data/areas-juridicas.ts:315-392.

FAQ Structure

Each legal area includes an array of `faqs`, which consist of `question` and `answer` strings. This data is used to render the `AreaFAQ` component and to generate `FAQPage` JSON-LD schemas for Google Search lib/data/areas-juridicas.ts:51-78.

Sources: lib/data/areas-juridicas.ts:1-392

Data Flow & System Integration

The `areasData` constant acts as the "Natural Language Space" source that is transformed into "Code Entity Space" objects throughout the Next.js lifecycle.

Code Entity Mapping

The following diagram illustrates how the raw data in `areas-juridicas.ts` is consumed by various system modules.

Title: Data Consumption Flow

```
graph TD
    subgraph "Natural Language Space (lib/data/areas-juridicas.ts)"
        DATA["areasData Object"]
        SLUGS["Keys: penal, civil, laboral..."]
    end

    subgraph "Code Entity Space"
        SITEMAP["app/sitemap.ts"]
        METADATA["lib/seo/metadata.ts"]
        DYNAMIC_ROUTE["app/areas-juridicas/[slug]/page.tsx"]
        UI_COMP["components/AreasJuridicas.tsx"]
    end

    DATA -->|Object.values| SITEMAP
    DATA -->|Lookup by Slug| METADATA
    DATA -->|Props Drilling| DYNAMIC_ROUTE
```

```
    DATA -->|Mapping| UI_COMP
    SLUGS -->|Params| DYNAMIC_ROUTE
```

Sources: lib/data/areas-juridicas.ts:2-3, app/sitemap.ts:2-21, app/areas-juridicas/[slug]/page.tsx:1-20

Key Functions

getAreaBranches()

While not explicitly exported as a standalone utility in the provided snippets, the logic for extracting branches is embedded within the `AreaData` structure. Each area defines a `branches` array (e.g., `['Defensa penal integral', 'Recursos penales']`) which is used by the `AreaDetailBranches` component to render the service grid lib/data/areas-juridicas.ts:44-50.

Sitemap Generation

The `sitemap.ts` file dynamically iterates over the `areasData` to generate the XML sitemap. This ensures that any new legal area added to the data model is automatically indexed by search engines without manual configuration.

```
// app/sitemap.ts logic
const areaPages = Object.values(areasData).map((area) => ({
  url: `${baseUrl}/areas-juridicas/${area.id}`,
  lastModified: new Date(),
  changeFrequency: 'monthly',
  priority: 0.8,
}));
```

Sources: app/sitemap.ts:15-21

SEO and UI Driving Logic

The data model is specifically designed to support the firm's SEO strategy. Each entry in `areasData` contains specific fields that map directly to HTML `<head>` elements and Structured Data.

Metadata Mapping

AreaData Field	Usage in System
`metaTitle`	Injected into `generateAreaMetadata` for the `<title>` tag lib/data/areas-juridicas.ts:12.
`metaDescription`	Used for the `description` meta tag and OpenGraph/Twitter cards lib/data/areas-juridicas.ts:13-14.
`metaKeywords`	Combined with base keywords to populate the `keywords` meta tag lib/data/areas-juridicas.ts:15-28.
`image`	Used as the OG Image and Hero background for the specific area lib/data/areas-juridicas.ts:29.

UI Rendering Architecture

The dynamic route `app/areas-juridicas/[slug]/page.tsx` uses the `slug` parameter to fetch the corresponding object from `areasData`. This object is then passed to specialized components:

Title: UI Component Data Binding

```
graph LR
    subgraph "AreaData Entry"
        TITLE["title"]
        SUB["subtitle"]
        DESC["longDescription"]
        SRV["services"]
        FAQ["faqs"]
    end

    subgraph "UI Components"
        HERO["AreaDetailHero"]
        SPLIT["AreaContentSplit"]
```

```
        WHY["AreaWhyChoose"]
        FAQ_UI["AreaFAQ"]
    end

    TITLE --> HERO
    SUB --> HERO
    DESC --> SPLIT
    SRV --> WHY
    FAQ --> FAQ_UI
```

Sources: lib/data/areas-juridicas.ts:4-78, app/sitemap.ts:16-21, app/robots.ts:3-36

Utility Functions

This page documents the core utility layer of the LegalOrbis codebase. It focuses on the `cn()` helper function located in `lib/utils.ts`, which manages conditional styling and Tailwind CSS class merging, and the TypeScript path alias configuration that facilitates clean imports across the project.

Class Name Merging (`cn`)

The `cn` utility is a foundational helper used in virtually every UI component within the repository. It provides a robust way to conditionally apply CSS classes while resolving conflicts between Tailwind CSS utility classes.

Implementation

The function leverages two industry-standard libraries: 1. **`clsx`**: A tiny utility for constructing `className` strings conditionally. 2. **`tailwind-merge`**: Specifically designed to handle Tailwind class overrides (e.g., ensuring `px-4` is correctly overridden by `px-6` if both are provided).

The implementation is defined as follows: lib/utils.ts:1-7

Data Flow and Logic

When a component calls `cn()`, the following process occurs:

1. **Input Gathering**: The function accepts a variadic list of `ClassValue` types (strings, objects, arrays, or falsy values).
2. **Conditional Resolution**: `clsx` evaluates the inputs. Falsy values (null, undefined, false) are discarded, and object keys are included only if their values are truthy.
3. **Conflict Resolution**: The resulting string is passed to `twMerge`. This step is critical for components that accept a `className` prop from a parent. `twMerge` ensures that the last class defined for a specific CSS property wins, preventing the "specificity war" common in standard string concatenation.

Usage Example in UI Primitives

The `cn` utility is used extensively in the `components/ui/` layer to allow for style overrides. For example, in the `Button` component: components/ui/button.tsx:40-45 (Hypothetical reference based on standard project structure)

Sources: - lib/utils.ts:1-7

TypeScript Path Aliases

LegalOrbis uses TypeScript path mapping to avoid "relative import hell" (e.g., `../../../components`). This is configured in the project root to ensure consistency across the `app/`, `components/`, and `lib/` directories.

Configuration

The `@/*` alias is mapped to the root directory `./*`. This allows any file within the project to be referenced relative to the root using the `@` prefix.

tsconfig.json:25-29

Import Mapping Diagram

The following diagram illustrates how the path alias maps "Natural Language Space" (logical project structure) to "Code Entity Space" (file system paths).

Path Alias Resolution Flow

```
graph TD
    subgraph "Logical Import (Natural Language Space)"
        A["@/components/ui/button"]
        B["@/lib/utils"]
        C["@/types/to-ico"]
    end

    subgraph "tsconfig.json Mapping Logic"
        M1["'@/*' -> './*'"]
    end

    subgraph "Physical File (Code Entity Space)"
        F1["./components/ui/button.tsx"]
        F2["./lib/utils.ts"]
        F3["./types/to-ico.d.ts"]
    end

    A --> M1
    B --> M1
    C --> M1
    M1 --> F1
    M1 --> F2
    M1 --> F3
```

Sources: - tsconfig.json:25-29

Integration with Custom Type Definitions

The utility and configuration layer also interacts with custom ambient type definitions. For instance, the project includes specific type declarations for the `to-ico` library used in the favicon generation pipeline. These types are automatically discovered by the TypeScript compiler because the `types/**/*.d.ts` pattern is included in the `tsconfig.json` configuration.

tsconfig.json:36-36 types/to-ico.d.ts:1-10

System Dependency Diagram

This diagram shows how the `cn` utility and path aliases serve as the "glue" between UI components and the underlying configuration.

Utility Dependency Architecture

```
graph LR
    subgraph "UI Layer"
        Component["React Component (.tsx)"]
    end

    subgraph "Utility Layer"
        CN["cn() Helper"]
        TailwindMerge["tailwind-merge"]
        Clsx["clsx"]
    end

    subgraph "Configuration Layer"
        TSConfig["tsconfig.json (Path Aliases)"]
        Globals["app/globals.css"]
    end

    Component -- "Uses for styling" --> CN
    CN -- "Wraps" --> TailwindMerge
    CN -- "Wraps" --> Clsx
    Component -- "Resolves via" --> TSConfig
    Component -- "Applies classes from" --> Globals
```

Sources: - lib/utils.ts:1-7 - tsconfig.json:25-29 - types/to-ico.d.ts:1-10

SEO & Structured Data

The LegalOrbis SEO architecture is designed to maximize visibility for legal services in the Madrid region. It leverages Next.js 15 metadata APIs, automated JSON-LD schema generation, and dynamic sitemap construction to ensure that every practice area is indexed with high precision.

System Overview

The SEO subsystem is divided into three primary functional areas: 1. **Metadata Management**: Dynamic generation of HTML tags (title, description, OpenGraph, Twitter) for every route. 2. **Structured Data**: Generation of JSON-LD schemas to enable Google Rich Results (LocalBusiness, LegalService, FAQ, etc.). 3. **Crawler**

Orchestration: Automated generation of `sitemap.xml` and `robots.txt` based on the application's data layer.

SEO Data Flow

The following diagram illustrates how the system transforms internal configuration and practice area data into SEO-compliant output.

SEO Generation Architecture

```
graph TD
    subgraph "Code Entity Space (lib/)"
        A["siteConfig (metadata.ts)"]
        B["areasData (areas-juridicas.ts)"]
        C["generateBaseMetadata()"]
        D["generateAreaMetadata()"]
        E["generateOrganizationSchema()"]
    end

    subgraph "Next.js Rendering Space (app/)"
        F["Root Layout (layout.tsx)"]
        G["Area Page (page.tsx)"]
        H["Robots Route (robots.ts)"]
        I["Sitemap Route (sitemap.ts)"]
    end

    A --> C
    B --> D
    C --> F
    D --> G
    E --> F
    B --> I
    I --> H
```

Sources: lib/seo/metadata.ts:4-36, lib/seo/schema.ts:121-196, app/layout.tsx:25-33, app/sitemap.ts:15-21.

Metadata Generation

Metadata is centralized in `lib/seo/metadata.ts`. The system uses a `siteConfig` object as a single source of truth for global values like the site name, base URL, and default keywords lib/seo/metadata.ts:4-36.

- **Global Metadata**: The `generateBaseMetadata()` function is invoked in the root `layout.tsx` to set default titles, OpenGraph tags, and favicon definitions lib/seo/metadata.ts:39-97.
- **Page-Specific Metadata**: The `generatePageMetadata()` utility handles logic for description truncation (max 160 characters) and keyword merging lib/seo/metadata.ts:100-149.
- **Practice Area Factory**: `generateAreaMetadata()` provides a specialized factory for dynamic routes under `/areas-juridicas/[slug]`, automatically appending location-based keywords like "Madrid" to service titles lib/seo/metadata.ts:152-181.

For details, see Metadata Generation.

JSON-LD Structured Data

Structured data is generated via a suite of TypeScript functions in `lib/seo/schema.ts`. These functions return objects compliant with `Schema.org` standards, which are then injected into the `<head>` of pages using `<script type="application/ld+json">`.

Schema Type	Purpose	Implementation
Organization	Defines the firm's identity and logo.	`generateOrganizationSchema()` lib/seo/schema.ts:121
LocalBusiness	Provides office location, phone, and geo-coordinates.	`generateLocalBusinessSchema()` lib/seo/schema.ts:199
LegalService	Specific details for Penal, Civil, etc.	`generateLegalServiceSchema()` lib/seo/schema.ts:234

Schema Type	Purpose	Implementation
Breadcrumb	Enhances SERP snippets with navigation paths.	`generateBreadcrumbSchema()` lib/seo/schema.ts:265
FAQ	Enables collapsible FAQ snippets in search results.	`generateFAQSchema()` lib/seo/schema.ts:283

For details, see JSON-LD Structured Data Schemas.

Robots & Sitemap

The application uses Next.js dynamic route handlers to generate crawler instructions.

- **Sitemap**: `app/sitemap.ts` dynamically iterates over the `areasData` object to generate URLs for every practice area, assigning a priority of `0.8` to area pages and `1.0` to the homepage app/sitemap.ts:8-23.
- **Robots**: `app/robots.ts` configures crawler rules, explicitly allowing access to optimized Next.js images and favicon assets while blocking private paths like `/admin/` or `/_next/server/` app/robots.ts:7-35.

Crawler Instruction Mapping

```
graph LR
    subgraph "Natural Language Space"
        S["Search Engine Crawler"]
        P["Practice Areas"]
        L["Legal Office Location"]
    end

    subgraph "Code Entity Space"
        S -->|"Reads"| R["app/robots.ts"]
        R -->|"Points to"| SM["app/sitemap.ts"]
        SM -->|"Fetches Slugs"| AD["lib/data/areas-juridicas.ts"]
        P --> AD
        L --> GS["generateLocalBusinessSchema()"]
    end
```

Sources: app/robots.ts:1-37, app/sitemap.ts:1-25, lib/seo/schema.ts:199-231.

For details, see Robots & Sitemap.

Metadata Generation

The metadata generation system in LegalOrbis provides a centralized, type-safe approach to managing Search Engine Optimization (SEO) tags across the application. It utilizes the Next.js Metadata API to generate standard meta tags, OpenGraph data, Twitter cards, and favicon references dynamically based on the page context.

Core Configuration

The foundation of the SEO system is the `siteConfig` object, which contains the global brand identity and default values used across the entire site.

Property	Value / Purpose
`name`	"Legal Orbis Abogados" - Used in title templates.
`url`	`https://www.legalorbisabogados.es` - Base URL for canonicals.
`ogImage`	`/LegalOrbis.svg` - Default social sharing image.
`keywords`	Array of 10 primary legal service keywords in Madrid.
`robots`	Configured to `index: true, follow: true` with specific GoogleBot snippet rules.

Sources: lib/seo/metadata.ts:4-36

System Architecture

The metadata system follows a hierarchical pattern where base metadata is defined at the root and specialized functions extend or override these values for specific pages.

Metadata Flow Diagram

"Metadata Generation Logic"

```
graph TD
    subgraph "Natural Language Space"
        A["Brand Identity"]
        B["Practice Area Content"]
        C["SEO Best Practices"]
    end

    subgraph "Code Entity Space"
        SC["siteConfig (Object)"]
        GBM["generateBaseMetadata()"]
        GPM["generatePageMetadata()"]
        GAM["generateAreaMetadata()"]
        RL["RootLayout (app/layout.tsx)"]
        AP["Area Page (app/areas-juridicas/[slug]/page.tsx)"]
    end

    A --> SC
    SC --> GBM
    GBM --> RL

    B --> GAM
    GAM --> GPM
    GPM --> AP

    C --> GPM
```

Sources: lib/seo/metadata.ts:4-181, app/layout.tsx:25-25

Implementation Details

Base Metadata

The `generateBaseMetadata()` function creates the initial configuration used in the root layout. It establishes the title template `%s | Legal Orbis Abogados`, which allows child pages to inject their own title while maintaining brand consistency.

Key features include: * **Metadata Base:** Sets the root URL for resolving relative asset paths lib/seo/metadata.ts:41-41. * **Icon Suite:** Defines a comprehensive set of favicons including `.ico`, `.svg`, and various `.png` sizes for modern browsers and Apple devices lib/seo/metadata.ts:74-89. * **OpenGraph/Twitter:** Standardizes the locale (`es_ES`) and card types (`summary_large_image`) lib/seo/metadata.ts:52-73.

Sources: lib/seo/metadata.ts:39-97, app/layout.tsx:25-25

Page Metadata Factory

The `generatePageMetadata()` function serves as a utility for individual routes. It implements specific logic to ensure SEO compliance:

- **Description Truncation:** If a description exceeds 160 characters, it is truncated to 157 characters followed by an ellipsis (`...`) to fit within Google search result limits lib/seo/metadata.ts:114-117.
- **Keyword Merging:** It automatically merges page-specific keywords with the global `siteConfig.keywords` array lib/seo/metadata.ts:122-122.
- **Canonical Resolution:** Dynamically constructs the canonical URL by appending the provided path to the base site URL lib/seo/metadata.ts:146-146.

Sources: lib/seo/metadata.ts:100-149

Legal Area Metadata

The `generateAreaMetadata()` function is a specialized factory for practice area detail pages. It follows a specific SEO pattern for the legal industry:

1. **Title Pattern:** `Abogados [Area] en Madrid | Legal Orbis` lib/seo/metadata.ts:165-165.

2. **Description Pattern:** `[Description] Especialistas en [area] en Madrid. Consulta gratuita.` lib/seo/metadata.ts:166-166.
3. **Keyword Injection:** Automatically generates localized keywords like `derecho [area] Madrid` and `defensa [area] Madrid` lib/seo/metadata.ts:171-177.

Sources: lib/seo/metadata.ts:152-181

Data Transformation Mapping

The following diagram illustrates how raw input data is transformed into standardized SEO metadata through the internal functions.

"Metadata Transformation Mapping"

```
graph LR
    subgraph "Input Data"
        ID_Title["areaTitle: 'Penal'"]
        ID_Desc["areaDescription: 'Defensa en delitos...'"]
    end

    subgraph "lib/seo/metadata.ts"
        GAM["generateAreaMetadata()"]
        GPM["generatePageMetadata()"]
    end

    subgraph "Final Metadata Object"
        MT_Title["title: 'Abogados Penal en Madrid | Legal Orbis'"]
        MT_Desc["description: 'Defensa en delitos... Especialistas en penal en Madrid...'"]
        MT_Canon["alternates.canonical: '.../penal'"]
    end

    ID_Title --> GAM
    ID_Desc --> GAM
    GAM -- "Constructs specialized strings" --> GPM
    GPM -- "Truncates & Merges" --> MT_Title
    GPM --> MT_Desc
    GPM --> MT_Canon
```

Sources: lib/seo/metadata.ts:100-181

Summary of Metadata Properties

Function	Title Logic	Description Logic	Canonical Logic
`generateBaseMetadata`	Uses `siteConfig.name`	Uses `siteConfig.description`	`siteConfig.url`
`generatePageMetadata`	`title | siteConfig.name`	Truncated to 160 chars	`siteConfig.url + url`
`generateAreaMetadata`	`Abogados [Area] en Madrid...`	Appends "Especialistas en..."	`siteConfig.url + areaUrl`

Sources: lib/seo/metadata.ts:39-181

JSON-LD Structured Data Schemas

This page documents the implementation of structured data schemas within the LegalOrbis codebase. These schemas use the JSON-LD format to provide search engines with explicit information about the firm's organization, physical location, legal services, and site structure, enhancing Search Engine Optimization (SEO) and enabling rich snippets in search results.

Implementation Overview

The system centralizes schema generation in `lib/seo/schema.ts`. It defines five primary generators that produce objects conforming to Schema.org specifications. These objects are then injected into the HTML `<head>` or `<body>` using `<script type="application/ld+json">` tags.

Core Architecture

The data flow starts from the page-level components (Root Layout or Dynamic Area Pages), which call the generator functions and serialize the output.

Data Flow: Schema Generation to Injection

```
graph TD
    subgraph "Data Source"
        A["areasData (lib/data)"]
    end

    subgraph "Schema Generators (lib/seo/schema.ts)"
        B["generateOrganizationSchema"]
        C["generateLocalBusinessSchema"]
        D["generateLegalServiceSchema"]
        E["generateBreadcrumbSchema"]
        F["generateFAQSchema"]
    end

    subgraph "Injection Points"
        G["Root Layout (app/layout.tsx)"]
        H["Area Detail Page (app/areas-juridicas/[slug]/page.tsx)"]
    end

    A --> D
    A --> F

    B --> G
    C --> G

    D --> H
    E --> H
    F --> H
```

Sources: lib/seo/schema.ts:1-290, app/layout.tsx:32-56, app/areas-juridicas/[slug]/page.tsx:60-103

Schema Generators

Global Schemas (Root Layout)

These schemas are injected globally via the `RootLayout` to represent the firm's identity and physical presence.

Function	Interface	Description
`generateOrganizationSchema`	`OrganizationSchema`	Defines the firm as a `LegalService`, `Attorney`, and `Organization`. Includes logo, contact points, and a service catalog.
`generateLocalBusinessSchema`	`LocalBusinessSchema`	Provides geographic coordinates (lat/long), opening hours, and address for local SEO.

- **Organization Schema:** Includes an `OfferCatalog` listing the main practice areas (Penal, Civil, Laboral, etc.) lib/seo/schema.ts:157-194.
- **LocalBusiness Schema:** Specifically targets the Madrid office location with `GeoCoordinates` lib/seo/schema.ts:216-220.

Sources: lib/seo/schema.ts:121-231, app/layout.tsx:32-33

Contextual Schemas (Dynamic Pages)

These generators are used within `app/areas-juridicas/[slug]/page.tsx` to provide detail-specific metadata.

LegalService Schema

Generated via `generateLegalServiceSchema`. It describes a specific legal practice area (e.g., "Derecho Penal"). * **Provider:** Links back to the main organization lib/seo/schema.ts:247-251. * **Offers:** Includes a standard "First consultation free" description lib/seo/schema.ts:257-260.

Breadcrumb Schema

Generated via `generateBreadcrumbSchema`. It maps the navigation path: `Inicio > Áreas Jurídicas > [Area Name]`. * **Positioning:** Uses `itemListElement` with incremental positions lib/seo/schema.ts:273-278.

FAQ Schema

Generated via `generateFAQSchema` only if the specific legal area contains FAQ data. * **Structure:** Maps `area.faqs` to `Question` and `Answer` types lib/seo/schema.ts:283-290.

Sources: lib/seo/schema.ts:234-290, app/areas-juridicas/[slug]/page.tsx:61-80

Technical Interfaces

The schemas are backed by TypeScript interfaces to ensure compliance with JSON-LD standards.

Class Diagram: Schema Type Definitions

```
classDiagram
    class OrganizationSchema {
        +String @context
        +String[] @type
        +String name
        +String url
        +Object address
        +Object contactPoint
        +Object hasOfferCatalog
    }
    class LocalBusinessSchema {
        +String @type
        +String telephone
        +Object geo
        +String[] openingHours
    }
    class LegalServiceSchema {
        +String name
        +String description
        +Object provider
        +String[] serviceType
    }
    class BreadcrumbSchema {
        +Object[] itemListElement
    }
    class FAQSchema {
        +Object[] mainEntity
```

```
    }

    OrganizationSchema <|-- LocalBusinessSchema : "Inherits Context"
```

Sources: lib/seo/schema.ts:3-118

Injection Mechanism

The application uses Next.js Server Components to inject these schemas. Since they are static JSON objects, they are serialized using `JSON.stringify()` and placed inside a script tag with `dangerouslySetInnerHTML`.

Global Injection (`app/layout.tsx`)

The `Organization` and `LocalBusiness` schemas are placed in the `<head>` of the root layout to ensure they are present on every page of the site. app/layout.tsx:45-56

Page-Specific Injection (`app/areas-juridicas/[slug]/page.tsx`)

The `LegalService`, `Breadcrumb`, and `FAQ` schemas are injected at the top of the `<main>` element within the dynamic route. app/areas-juridicas/[slug]/page.tsx:84-103

Sources: app/layout.tsx:45-56, app/areas-juridicas/[slug]/page.tsx:84-103

Robots & Sitemap

This section documents the automated search engine optimization (SEO) infrastructure of the LegalOrbis application. It covers the dynamic generation of the `robots.txt` file and the `sitemap.xml` file, which together guide web crawlers in indexing the site effectively.

Robots Configuration (`robots.ts`)

The file `app/robots.ts` utilizes the Next.js `MetadataRoute.Robots` type to define crawling rules for search engines. It ensures that essential assets are accessible while protecting private or internal server routes from indexing.

Implementation Details

The configuration specifies a `baseUrl` of `https://www.legalorbisabogados.es` app/robots.ts:4-4 and defines a set of rules applied to all user agents (`*`) app/robots.ts:8-8.

Allow Rules: * **Static Assets:** Explicitly allows `/_next/static/` and `/_next/image` to ensure Google and other crawlers can render the page correctly using optimized images app/robots.ts:11-13. * **Favicon v2:** Specifically allows the versioned favicon assets (e.g., `favicon-v2.ico` , `favicon-48x48-v2.png`) generated by the asset pipeline app/robots.ts:15-17. * **Legacy Assets:** Allows access to legacy favicon paths and the `/images/` directory to follow redirects and index original media app/robots.ts:19-24.

Disallow Rules: * **Internal Routes:** Blocks access to `/api/` , `/admin/` , `/private/` , and the Next.js internal server directory `/_next/server/` app/robots.ts:27-32.

The function also provides the absolute URL to the sitemap app/robots.ts:34-34.

Sources: * app/robots.ts:1-36

Sitemap Generation (`sitemap.ts`)

The `app/sitemap.ts` file dynamically generates the site map by combining static routes with dynamic routes derived from the legal practice areas data.

Data Flow and Logic

The sitemap is constructed using the `areasData` object from the application's data layer app/sitemap.ts:2-2.

1. **Home Page:** Defined with the highest priority (`1.0`) and a `monthly` change frequency app/sitemap.ts:8-13.
2. **Dynamic Area Pages:** The function iterates over `Object.values(areasData)` to generate URLs for every practice area defined in the system (e.g., `/areas-juridicas/penal`, `/areas-juridicas/civil`) app/sitemap.ts:16-17.
3. **Attributes:** Each dynamic page is assigned a priority of `0.8` and a `monthly` change frequency app/sitemap.ts:19-20.

Sitemap Structure Diagram

The following diagram illustrates how the `sitemap()` function transforms internal data into the XML structure.

Sitemap Data Transformation

```
graph TD
    subgraph "Data Source"
        A["areasData (lib/data/areas-juridicas.ts)"]
    end

    subgraph "Sitemap Logic (app/sitemap.ts)"
        B["sitemap() function"]
        C["Static Route: /"]
        D["Dynamic Routes: /areas-juridicas/[id]"]
    end

    subgraph "Output (sitemap.xml)"
        E["URL: / (Priority 1.0)"]
        F["URL: /areas-juridicas/penal (Priority 0.8)"]
        G["URL: /areas-juridicas/civil (Priority 0.8)"]
        H["...others"]
    end

    A -- "Object.values()" --> B
    B --> C
    B --> D
    C --> E
    D --> F
```

```
    D --> G
    D --> H
```

Sources: * app/sitemap.ts:1-25 * lib/data/areas-juridicas.ts:2-80

Crawler Interaction Flow

This diagram maps the interaction between external Search Engine Crawlers and the code entities responsible for guiding them.

Crawler Guidance Flow

```
sequenceDiagram
    participant Crawler as "Search Engine Crawler"
    participant Robots as "app/robots.ts"
    participant Sitemap as "app/sitemap.ts"
    participant Data as "lib/data/areas-juridicas.ts"

    Crawler->>Robots: GET /robots.txt
    Robots-->>Crawler: Return rules & Sitemap URL
    Crawler->>Sitemap: GET /sitemap.xml
    Sitemap->>Data: Read areasData keys
    Data-->>Sitemap: Return area IDs (penal, civil, etc.)
    Sitemap-->>Crawler: Return XML with all dynamic URLs
```

Configuration Summary

Feature	Code Entity	Configuration / Value
Base URL	`baseUrl`	`https://www.legalorbisabogados.es`
Root Priority	`homePage.priority`	`1.0`
Area Priority	`areaPages.priority`	`0.8`
Change Freq	`changeFrequency`	`monthly`

Feature	Code Entity	Configuration / Value
Allowed Paths	`robots.rules.allow`	`/_next/static/`, `/_next/image`, `/favicon-v2.ico`, etc.
Blocked Paths	`robots.rules.disallow`	`/api/`, `/admin/`, `/_next/server/`

Sources: * app/robots.ts:4-35 * app/sitemap.ts:5-23

Styling & Theming

LegalOrbis utilizes a modern styling stack centered around **Tailwind CSS 4** and **CSS Custom Properties**. The system is designed for high performance, utilizing a combination of CSS-native animations and Framer Motion for interactive elements. The brand identity is codified into a specific teal-based color palette that adapts to both light and dark modes.

Architecture Overview

The styling architecture is divided into three primary layers: 1. **Tokens & Variables**: Definition of the brand color palette and system spacing using CSS variables. 2. **Tailwind Integration**: Configuration of the utility engine to map these variables to functional classes. 3. **Utility Components**: High-level CSS classes for recurring design patterns (e.g., gradients, buttons).

Visual Styling Flow

The following diagram illustrates how raw CSS variables flow through the Tailwind engine to become usable utility classes in React components.

Styling Pipeline: Tokens to UI

```
graph TD
    subgraph "CSS Variable Layer [app/globals.css]"
```

```
        V1["--primary: 180 45% 25%"]
        V2["--background: 0 0% 100%"]
        V3["--radius: 0.5rem"]
    end

    subgraph "Tailwind Engine [tailwind.config.ts]"
        T1["primary: hsl(var(--primary))"]
        T2["bg-background"]
        T3["rounded-lg: var(--radius)"]
    end

    subgraph "Component Layer [React]"
        C1["legal-button"]
        C2["section-padding"]
        C3["container-max"]
    end

    V1 --> T1
    V2 --> T2
    V3 --> T3
    T1 --> C1
    T2 --> C2
    T3 --> C3
```

Sources: app/globals.css:46-80, tailwind.config.ts:14-74, app/globals.css:162-176

Brand Color System

The "Legal Orbis Teal" is the core of the visual identity. It is implemented using HSL values to allow for easy opacity manipulation and dark mode adjustments.

Token Name	Hex / Value	Role
`legal-primary`	`#1a5f5f`	Main brand teal (Dark)
`legal-secondary`	`#5eb4d4`	Bright accent blue
`legal-accent`	`#2d7a7a`	Medium teal for UI elements
`legal-dark`	`#0d3d3d`	Deep teal for high contrast
`--primary`	`180 45% 25%`	Primary button/link color (Light Mode)
`--primary`	`180 45% 35%`	Primary button/link color (Dark Mode)

Sources: tailwind.config.ts:16-22, app/globals.css:54-54, app/globals.css:89-89

Global Styles & Custom Utilities

Beyond standard Tailwind utilities, the codebase defines a set of "Legal Orbis" specific classes in the `@layer components` section of the global stylesheet. These classes encapsulate complex styles like the brand gradient and standard section layouts.

- **Gradients**: `.legal-gradient` and `.legal-text-gradient` use the brand teal-to-blue transition app/globals.css:142-151.
- **Layout Primitives**: `.section-padding` (standardized vertical spacing) and `.container-max` (max-width of 1600px) ensure consistency across the landing page and detail views app/globals.css:170-176.
- **Buttons**: `.legal-button` and `.legal-button-secondary` provide pre-styled interactive elements that follow the brand's rounded-corner and transition specs app/globals.css:162-168.

For a full list of variables and utility classes, see **Global Styles & CSS Variables**.

Animation Strategy

LegalOrbis uses a dual-track animation strategy to balance performance and interactivity:

1. **CSS-Native Animations**: Defined in `app/globals.css`, these use `@keyframes` (like `fadeInUp` and `fadeInLeft`) for scroll-triggered entrance effects. They are orchestrated by the `CSSAnimatedSection` component app/globals.css:197-255.
2. **Framer Motion**: Used for complex state-driven animations, such as the `HeroCarousel` or the `QuienesSomos` dot navigation, where JS-based control is required.

Animation Logic Association

```
graph LR
    subgraph "CSS Space [app/globals.css]"
        A1["@keyframes fadeInUp"]
        A2["@keyframes fadeInLeft"]
        A3[".animate-fadeInUp"]
```

```
    end

    subgraph "Logic Space [components/ui/]"
        B1["CSSAnimatedSection.tsx"]
        B2["AnimatedSection.tsx (Framer)"]
    end

    subgraph "Consumer Space [components/]"
        C1["VideoBanner"]
        C2["HeroCarousel"]
    end

    A1 --> A3
    A3 --> B1
    B1 --> C1
    B2 --> C2
```

Sources: app/globals.css:197-243, components/video-banner.tsx:30-45, tailwind.config.ts:75-93

For details on PostCSS integration and animation configuration, see **Tailwind Configuration & Animation**.

Global Styles & CSS Variables

This section documents the foundational styling architecture of LegalOrbis, centered around `app/globals.css`. The project utilizes Tailwind CSS 4 and CSS custom properties (variables) to maintain a consistent brand identity across light and dark modes.

CSS Custom Properties (Design Tokens)

The application uses CSS variables to define its design tokens, which are then mapped to Tailwind utility classes. This approach allows for dynamic theme switching and centralized management of brand colors.

Root Tokens (Light Mode)

The default theme is based on a "Legal Orbis Teal" palette.

Variable	Value	Description
`--primary`	`180 45% 25%`	Primary brand teal used for buttons and highlights app/globals.css:54.
`--background`	`0 0% 100%`	Main application background (White) app/globals.css:48.
`--foreground`	`0 0% 15%`	Primary text color (Dark Grey/Black) app/globals.css:49.
`--accent`	`180 45% 25%`	Same as primary, used for interactive elements app/globals.css:60.
`--radius`	`0.5rem`	Base border radius for components app/globals.css:47.

Dark Mode Variants

Dark mode is activated via the `.dark` class. It shifts the primary teal to a lighter shade for better contrast against dark backgrounds.

Variable	Value	Description
`--background`	`0 0% 15%`	Dark background app/globals.css:83.
`--primary`	`180 45% 35%`	Lighter teal for accessibility in dark mode app/globals.css:89.
`--border`	`0 0% 100% / 10%`	Semi-transparent white borders app/globals.css:99.

Sources: app/globals.css:46-115

Tailwind Integration & Theme Mapping

LegalOrbis uses the `@theme inline` directive to map CSS variables directly into the Tailwind engine. This ensures that utility classes like `bg-primary` or `text-foreground` stay in sync with the custom properties defined above.

Design Token Flow

The following diagram illustrates how raw CSS values flow from the `:root` definition into the React component layer.

Token Resolution Diagram

```
graph TD
    subgraph "CSS Variable Layer (app/globals.css)"
        V1["--primary: 180 45% 25%"]
        V2["--background: 0 0% 100%"]
    end

    subgraph "Tailwind Theme Mapping (@theme)"
        T1["--color-primary: var(--primary)"]
        T2["--color-background: var(--background)"]
    end

    subgraph "React Component Layer"
        C1["className='bg-primary'"]
        C2["className='text-background'"]
    end

    V1 --> T1
    V2 --> T2
    T1 --> C1
    T2 --> C2
```

Sources: app/globals.css:6-44, app/globals.css:46-80

Custom Utility Classes

Beyond standard Tailwind utilities, the codebase defines several "Legal Orbis" specific classes under the `@layer components` directive to encapsulate complex or repeated styles.

Layout & Spacing

- `.container-max`: Sets a maximum width of `1600px` and centers the content. Used for high-resolution displays app/globals.css:174-176.
- `.section-padding`: Standardizes vertical spacing for page sections (`py-32`) and horizontal gutters app/globals.css:170-172.

UI Elements

- `.legal-gradient`: A 135-degree linear gradient from teal (`#1a5f5f`) to light blue (`#5eb4d4`) app/globals.css:142-144.
- `.legal-button`: The primary CTA style, applying `--primary` background and transition effects app/globals.css:162-164.
- `.legal-card`: Encapsulates background, rounding, and hover shadow transitions for practice area cards app/globals.css:158-160.

Typography & Effects

- `.legal-text-gradient`: Applies the brand gradient to text using `-webkit-background-clip: text` app/globals.css:146-151.
- `.line-clamp-4`: A utility for truncating long descriptions to exactly four lines app/globals.css:179-184.

Sources: app/globals.css:141-195

CSS Animations

The application includes a set of hardware-accelerated CSS animations that do not rely on JavaScript. These are primarily utilized by the `CSSAnimatedSection` component.

Keyframe Definitions

The following animations are defined with a default duration of `0.6s` and an `ease-out` timing function: * `fadeInUp`: Moves element from `translateY(30px)` to `0` app/globals.css:197-206. * `fadeInLeft`: Moves element from `translateX(-30px)` to `0` app/globals.css:208-217. * `fadeInRight`: Moves element from `translateX(30px)` to `0` app/globals.css:219-228. * `fadeInScale`: Scales element from `0.95` to `1` app/globals.css:230-239.

Implementation Example

In `VideoBanner`, these animations are applied to reveal content as the user interacts with the page.

Animation Application Diagram

```
graph LR
    subgraph "app/globals.css"
        KF["@keyframes fadeInUp"]
        AN["class .animate-fadeInUp"]
    end

    subgraph "components/video-banner.tsx"
        CAS["CSSAnimatedSection"]
        H2["h2 (Equipo)"]
    end

    KF --> AN
    AN --> CAS
    CAS -->|"Wraps"| H2
```

Sources: app/globals.css:197-255, components/video-banner.tsx:39-43

Global Resets & Base Styles

The `@layer base` section applies global defaults to HTML elements: * **Smooth Scrolling**: Enabled via `scroll-behavior: smooth` for anchor navigation app/globals.css:124-126. * **Font Smoothing**: Antialiasing is forced for better legibility of

the Geist font app/globals.css:134-137. * **Performance**: `will-change-auto` is applied to `img`, `video`, and `iframe` to optimize rendering app/globals.css:128-132.

Sources: app/globals.css:117-138

Tailwind Configuration & Animation

This section documents the styling and animation architecture of the LegalOrbis platform. The project utilizes **Tailwind CSS** for utility-first styling, integrated with **Geist** font variables and a specialized animation pipeline that balances high-performance CSS transitions with complex Framer Motion interactions.

Tailwind Configuration

The core styling engine is defined in `tailwind.config.ts`, which extends the default Tailwind theme to include the LegalOrbis brand identity and system-level design tokens.

Brand Palette & System Tokens

The configuration defines a custom `legal` color family and maps system tokens to CSS variables defined in the global stylesheet.

Token Category	Key	Value / Reference
Legal Orbis Brand	`legal-primary`	`#1a5f5f` (Teal)
	`legal-secondary`	`#5eb4d4` (Light Blue)
	`legal-accent`	`#2d7a7a` (Medium Teal)
	`legal-light`	`#a8d8e6` (Very Light Blue)
	`legal-dark`	`#0d3d3d` (Dark Teal)
System Colors	`primary`	`hsl(var(--primary))`

Token Category	Key	Value / Reference
	`background`	`hsl(var(--background))`
	`card`	`hsl(var(--card))`
Typography	`sans`	`var(--font-geist-sans)`
	`mono`	`var(--font-geist-mono)`

Sources: tailwind.config.ts:14-74

Optimization & Keyframes

The configuration includes performance optimizations and custom keyframes for basic UI transitions: * **Future Flags**: `hoverOnlyWhenSupported: true` is enabled to prevent hover state "sticking" on touch devices tailwind.config.ts:11-13. * **Keyframes**: Custom definitions for `fadeIn`, `slideUp`, and `carouselSlide` are provided for standard CSS animations tailwind.config.ts:80-93.

PostCSS & Font Integration

The project uses the `@tailwindcss/postcss` pipeline to process styles. Font integration is handled via CSS variables injected by Next.js font loaders, which are then referenced in the Tailwind theme:

1. **Geist Sans**: Referenced as `var(--font-geist-sans)` for the primary sans-serif stack tailwind.config.ts:72.
2. **Geist Mono**: Referenced as `var(--font-geist-mono)` for code or technical displays tailwind.config.ts:73.

Animation Architecture

LegalOrbis employs a dual-strategy for animations, choosing between standard CSS transitions and Framer Motion based on the complexity and performance requirements of the component.

Animation Strategy Comparison

Feature	`CSSAnimatedSection`	`LazyAnimatedSection`
Engine	Native CSS + Intersection Observer	Framer Motion (`motion.div`)
Performance	High (GPU accelerated, no JS runtime for animation)	Moderate (Requires Framer Motion runtime)
Control	Basic (Delay, Duration)	Advanced (Variants, Spring physics, Viewport tracking)
Usage	High-traffic landing pages	Complex interactive elements

CSS-Based Animations

The `CSSAnimatedSection` component uses the native `IntersectionObserver` API to trigger CSS classes when an element enters the viewport.

Data Flow: CSS Animation Trigger

```
graph TD
    subgraph "Browser Runtime"
        A["IntersectionObserver"] -- "entry.isIntersecting" --> B["setIsVisible(true)"]
    end

    subgraph "React Component: CSSAnimatedSection"
        B --> C["Render div"]
        C -- "isVisible ? animate-fadeInUp : opacity-0" --> D["Apply CSS Class"]
    end

    subgraph "Tailwind / CSS"
        D --> E["animation-delay: delay"]
        E --> F["animation-fill-mode: both"]
    end
```

Sources: components/ui/css-animated-section.tsx:21-52

Framer Motion Animations

The `LazyAnimatedSection` component provides a more robust set of variants (`fadeInUp` , `fadeInDown` , `fadeInLeft` , `fadeInRight` , `fadeInScale` , `fadeIn`) using

the `framer-motion` library. It utilizes the `whileInView` prop to trigger animations once with a specific viewport margin components/ui/lazy-animated-section.tsx:49-58.

Entity Mapping: Animation Variants

```
graph LR
    subgraph "Code Entity: animationVariants"
        V1["fadeInUp"]
        V2["fadeInScale"]
        V3["fadeInLeft"]
    end

    subgraph "Visual Behavior"
        V1 --> B1["y: 30 -> 0, opacity: 0 -> 1"]
        V2 --> B2["scale: 0.95 -> 1, opacity: 0 -> 1"]
        V3 --> B3["x: -30 -> 0, opacity: 0 -> 1"]
    end

    subgraph "Implementation: LazyAnimatedSection"
        L["LazyAnimatedSection"] -- "variants={animationVariants[animation]}" --> V1
        L -- "viewport={{ once: true, amount: 0.2 }}" --> VIEW["Viewport Trigger"]
    end
```

Sources: components/ui/lazy-animated-section.tsx:6-31, components/ui/lazy-animated-section.tsx:51-54

Key Implementation Details

Intersection Observer Configuration

Both components are configured to trigger slightly before the element is fully visible to ensure a smooth transition: * **CSSAnimatedSection**: Uses a `threshold` of `0.1` and a `rootMargin` of `-50px` components/ui/css-animated-section.tsx:30-31. * **LazyAnimatedSection**: Uses an `amount` of `0.2` and a `margin` of `0px 0px -50px 0px` components/ui/lazy-animated-section.tsx:54.

Animation Utility Classes

Standard animations defined in the Tailwind config include: * `animate-fade-in`: `fadeIn 0.5s ease-in-out` tailwind.config.ts:76. * `animate-slide-up`: `slideUp 0.5s`

`ease-out` tailwind.config.ts:77. * `animate-carousel-slide`: `carouselSlide 0.5s ease-in-out` tailwind.config.ts:78.

Sources: * `tailwind.config.ts` * `components/ui/css-animated-section.tsx` * `components/ui/lazy-animated-section.tsx`

Infrastructure & Deployment

This section provides an overview of the LegalOrbis infrastructure stack, focusing on the production deployment environment, build-time optimizations, and automated maintenance workflows. The application is designed as a high-performance static site deployed on **Vercel**, utilizing **Next.js** features for asset optimization and security.

System Architecture Overview

The following diagram illustrates the relationship between the configuration files and the deployment lifecycle:

Deployment & Configuration Flow

```
graph TD
    subgraph "Local Development"
        [scripts/generate-favicons.ts] -->|"Generates"| [public/favicon-*.png]
        [scripts/generate-favicons.ts] -->|"Generates"| [public/favicon.ico]
    end

    subgraph "CI/CD (GitHub Actions)"
        [.github/workflows/ci.yml] -->|"Runs"| [npm_run_lint]
        [.github/workflows/ci.yml] -->|"Runs"| [npm_run_build]
    end

    subgraph "Runtime Environment (Vercel)"
        [next.config.ts] -->|"Configures"| [Next_Server]
        [vercel.json] -->|"Configures"| [Vercel_Edge]
        [Next_Server] -->|"Serves"| [App_Router]
        [Vercel_Edge] -->|"Headers/Redirects"| [Browser]
    end
```

```
    [renovate.json] -->|"Updates"| [package.json]
```

Sources: .github/workflows/ci.yml:1-31, next.config.ts:3-203, vercel.json:1-43, CLAUDE.md:13-19

7.1 Next.js Configuration

The `next.config.ts` file serves as the primary engine for application-level behavior. It handles complex image optimization pipelines, security headers, and SEO-critical redirection logic.

Key responsibilities include: * **Image Optimization**: Supporting high-efficiency formats like AVIF and WebP next.config.ts:4-11. * **Performance**: Stripping `console.log` in production and optimizing package imports for heavy libraries like `framer-motion` and `lucide-react` next.config.ts:19-36. * **Security & Caching**: Implementing strict HTTP headers (X-Frame-Options, CSP for SVGs) and an immutable caching strategy for static assets next.config.ts:38-152. * **Routing Logic**: Enforcing HTTPS, canonical `www` domains, and sanitizing URLs next.config.ts:157-202.

For details, see Next.js Configuration.

Sources: next.config.ts:1-205

7.2 Vercel Deployment Configuration

The `vercel.json` file complements the Next.js configuration by defining Edge-level behaviors. It ensures that the hosting environment respects the application's routing preferences, such as `cleanUrls` and the absence of trailing slashes.

- **Global Routing**: Configuration for `cleanUrls: true` and `trailingSlash: false` vercel.json:2-3.

- **Edge Headers**: Redundant cache-control enforcement for critical brand assets like the `apple-touch-icon.png` to ensure high availability and performance vercel.json:5-42.

For details, see Vercel Deployment Configuration.

Sources: vercel.json:1-43

7.3 Favicon & Asset Generation

LegalOrbis uses a scripted approach to asset management to ensure consistency across different device resolutions and platforms.

Asset Pipeline Entity Map

```
graph LR
    subgraph "Source Space"
        SVG["public/favicon.svg"]
    end

    subgraph "Code Entity Space (Scripts)"
        Script["scripts/generate-favicons.ts"]
        Sharp["Sharp Library"]
        ToIco["to-ico Library"]
    end

    subgraph "Output Space (Public Assets)"
        ICO["public/favicon.ico"]
        PNGs["public/favicon-96x96.png"]
        Apple["public/apple-touch-icon.png"]
        Manifest["public/site.webmanifest"]
    end

    SVG --> Script
    Script --> Sharp
    Sharp --> PNGs
    Sharp --> Apple
    PNGs --> ToIco
    ToIco --> ICO
```

The system relies on `scripts/generate-favicons.ts` to transform a single source SVG into a full suite of icons CLAUDE.md:18. This ensures that the `favicon-96x96.png` and other variants are always up to date with the latest brand identity public/icon-96x96.png:1-26.

For details, see Favicon & Asset Generation.

Sources: scripts/generate-favicons.ts, public/icon-96x96.png:1-26, CLAUDE.md:18-18

7.4 CI/CD & Dependency Management

The project maintains a high standard of code health through automated workflows and dependency tracking.

- **GitHub Actions**: The `ci.yml` workflow automatically triggers on pushes to `main` and `dev` branches, executing `npm run lint` and `npm run build` to prevent regressions .github/workflows/ci.yml:3-30.
- **Automated Updates**: **Renovate Bot** is configured to manage dependencies, with specific grouping for `Next.js` and `React` packages to ensure compatibility renovate.json:17-24. It is set to automerge patch updates every Monday morning renovate.json:4-11.
- **Browser Compatibility**: The project targets modern browsers as defined in `.browserslistrc`, ensuring the CSS features of Tailwind 4 are supported.

For details, see CI/CD & Dependency Management.

Sources: .github/workflows/ci.yml:1-31, renovate.json:1-26, CLAUDE.md:13-19

Next.js Configuration

The `next.config.ts` file serves as the central orchestration point for the LegalOrbis application's runtime behavior, build-time optimizations, and infrastructure-level

security. It configures the Next.js compiler, image processing pipeline, HTTP headers, and URL routing logic to ensure high performance and SEO compliance.

Image Optimization

LegalOrbis utilizes the Next.js Image component with a highly optimized configuration to serve assets efficiently across varying device types.

- **Formats**: The system prioritizes `image/avif` followed by `image/webp` to provide superior compression over standard JPEG/PNG next.config.ts:5-5.
- **Sizing Grid**: A comprehensive range of `deviceSizes` (from 640 to 3840 pixels) and `imageSizes` (from 16 to 384 pixels) is defined to allow the server to generate precise srcsets for responsive images next.config.ts:6-7.
- **SVG Security**: SVGs are permitted but restricted via a strict Content Security Policy (CSP). They are executed in a `sandbox` with `script-src 'none'` to prevent XSS attacks through malicious image files next.config.ts:9-10.

Image Processing Flow

```
graph TD
    ["Source Image (SVG/JPG/PNG)"] --> ["Next.js Image Loader"]
    subgraph "Optimization Pipeline"
        ["Next.js Image Loader"] --> ["Format Conversion (AVIF/WebP)"]
        ["Format Conversion (AVIF/WebP)"] --> ["Resizing (deviceSizes/imageSizes)"]
    end
    ["Resizing (deviceSizes/imageSizes)"] --> ["Cached Optimized Asset"]
    ["Cached Optimized Asset"] --> ["Client Browser"]

    subgraph "Security Layer"
        ["SVG Source"] --> ["CSP Sandbox"]
        ["CSP Sandbox"] --> ["script-src 'none'"]
    end
```

Sources: next.config.ts:4-11

Compiler & Build Optimizations

The configuration includes several flags to reduce bundle size and improve developer experience.

- **Console Stripping**: In production environments, the compiler automatically removes all `console.log` statements to keep the client-side logs clean and slightly reduce bundle size next.config.ts:19-21.
- **Modular Imports**: For `lucide-react`, the config uses `modularizeImports` to transform member imports into direct file paths, preventing the inclusion of the entire icon library in the bundle next.config.ts:23-27.
- **Package Optimization**: The `experimental.optimizePackageImports` property is used for heavy libraries like `framer-motion`, `@radix-ui/react-accordion`, and `@radix-ui/react-dialog`. This ensures that only the specific components used are loaded by the browser next.config.ts:29-36.

Sources: next.config.ts:18-36

HTTP Security Headers

LegalOrbis implements a robust security posture through global HTTP headers applied to all routes next.config.ts:38-60.

Header	Value	Purpose
`X-Frame-Options`	`DENY`	Prevents clickjacking by forbidding the site from being rendered in an iframe.
`X-Content-Type-Options`	`nosniff`	Prevents the browser from interpreting files as a different MIME type than declared.
`Referrer-Policy`	`origin-when-cross-origin`	Limits referrer information sent to other sites.
`Permissions-Policy`	`camera=(), microphone=(), ...`	Disables access to sensitive hardware features by default.

Sources: next.config.ts:43-58

Caching Strategy

The configuration defines a tiered caching strategy based on the nature of the assets.

1. **Static Assets (Immutable)**: Assets in `/_next/static/`, `/images/`, and root-level branding files (favicons, SVGs) are served with a `max-age` of 1 year (31,536,000 seconds) and the `immutable` directive next.config.ts:80-150.
2. **SEO Metadata**: `sitemap.xml` and `robots.txt` are cached for 24 hours (86,400 seconds) to ensure crawlers receive updated routing information without overloading the server next.config.ts:61-78.

Cache Policy Association

```
graph LR
    ["nextConfig.headers()"] --> ["Immutable (1 Year)"]
    ["nextConfig.headers()"] --> ["Short-Term (24 Hours)"]

    ["Immutable (1 Year)"] --- ["/_next/static/**"]
    ["Immutable (1 Year)"] --- ["/images/**"]
    ["Immutable (1 Year)"] --- ["/favicon.ico"]
    ["Immutable (1 Year)"] --- ["/LegalOrbis.svg"]

    ["Short-Term (24 Hours)"] --- ["/sitemap.xml"]
    ["Short-Term (24 Hours)"] --- ["/robots.txt"]
```

Sources: next.config.ts:61-151

Redirect & Canonicalization Rules

The `redirects` function handles protocol enforcement, domain canonicalization, and URL sanitization next.config.ts:157-202.

- **HTTPS Enforcement**: Detects the `x-forwarded-proto: http` header (typically from a proxy/load balancer) and redirects to the secure `https://www.legalorbisabogados.es` equivalent next.config.ts:160-171.
- **WWW Canonicalization**: Redirects requests from the apex domain `legalorbisabogados.es` to the `www` subdomain to maintain SEO consistency next.config.ts:173-183.

- **URL Sanitization**: Redirects common problematic characters like `/$` or `/&` back to the home page to prevent 404s from malformed links next.config.ts:185-194.
- **Practice Areas Navigation**: Redirects the general `/areas-juridicas` path to the `#areas-juridicas` anchor on the home page, as the landing page for all areas is integrated into the main scroll view next.config.ts:196-200.

Sources: next.config.ts:157-201

Vercel Deployment Configuration

This section documents the platform-specific configuration for Vercel, which hosts the LegalOrbis application. It covers the routing behavior, caching strategies for static assets, and the Progressive Web App (PWA) manifest integration.

Routing and Clean URLs

The application is configured to provide a seamless URL experience by enforcing clean paths and consistent trailing slash behavior.

- **Clean URLs**: Enabled to allow accessing paths without the `.html` extension vercel.json:2-2.
- **Trailing Slashes**: Disabled (`false`) to ensure that requests to `/areas-juridicas/` are redirected to `/areas-juridicas` for SEO consistency vercel.json:3-3.

Redirects

While the `vercel.json` file contains an empty redirects array vercel.json:4-4, specific application redirects (such as the `/areas-juridicas` to `/#areas-juridicas` anchor jump) are typically handled at the Next.js configuration level or via client-side routing logic to ensure users are directed to the correct section of the single-page architecture when they attempt to access the practice areas listing.

Static Asset Caching

To optimize performance and reduce bandwidth consumption, LegalOrbis implements an aggressive caching strategy for brand assets and favicons. All favicon-related assets are served with an immutable cache header.

Cache Strategy Implementation

The configuration targets specific file patterns in the `public/` directory and applies a `Cache-Control` header with a TTL of one year (`31536000` seconds) and the `immutable` directive vercel.json:11-11.

Asset Type	Source Pattern	Cache Policy
Apple Touch Icon	`/apple-touch-icon.png`	`public, max-age=31536000, immutable`
Legacy Favicon	`/favicon.ico`	`public, max-age=31536000, immutable`
Vector Favicon	`/favicon.svg`	`public, max-age=31536000, immutable`
Sized PNGs	`/favicon-:size.png`	`public, max-age=31536000, immutable`

Sources: vercel.json:5-42

PWA Manifest and Icons

The application includes a Web App Manifest to enable PWA features, allowing users to "install" the site on mobile devices and desktops.

Web Manifest Configuration

The `site.webmanifest` file defines the application's identity and visual behavior when launched from a home screen.

- **Identity**: Defined as "Legal Orbis Abogados" with a short name of "Legal Orbis" public/site.webmanifest:2-3.
- **Theme**: The `theme_color` is set to `#1A3635` (matching the brand's primary teal) and the `background_color` is white public/site.webmanifest:24-25.

- **Display**: Set to `standalone` to remove browser UI elements when opened as an app public/site.webmanifest:26-26.

Icon Suite

The manifest references a suite of icons generated to support different device requirements, including maskable icons for Android public/site.webmanifest:5-23.

Deployment Data Flow: Asset Delivery

The following diagram illustrates how Vercel interprets the configuration to serve assets from the `public/` directory with the specified headers.

Asset Delivery Pipeline

```
graph TD
    subgraph "Vercel Edge Network"
        Request["HTTP Request"] --> Router["vercel.json Rules"]
        Router --> Headers["Apply Cache-Control"]
        Headers --> Response["HTTP Response (200 OK)"]
    end

    subgraph "Public Assets Space"
        Favicon["/favicon.ico"]
        Manifest["/site.webmanifest"]
        AppleIcon["/apple-touch-icon.png"]
    end

    Router -- "Matches /favicon.ico" --> Favicon
    Router -- "Matches /apple-touch-icon.png" --> AppleIcon
    Router -- "Default Routing" --> Manifest
```

Sources: vercel.json:1-43, public/site.webmanifest:1-29

Technical Entity Mapping

The configuration links platform-level settings to specific files within the repository's `public` directory.

Configuration to Entity Mapping

```
graph LR
    subgraph "vercel.json (Vercel Config)"
        CleanURLs["cleanUrls: true"]
        TSlash["trailingSlash: false"]
        HeaderRule["headers: source: /favicon-:size.png"]
    end

    subgraph "public/ (Physical Assets)"
        Icon96["icon-96x96.png"]
        Fav96["favicon-96x96.png"]
        WebManifest["site.webmanifest"]
    end

    HeaderRule -.-> Fav96
    HeaderRule -.-> Icon96
    WebManifest -- "References" --> Fav96
```

Manifest Icon Definitions

The manifest explicitly maps sizes to specific file paths to ensure the browser selects the optimal resolution for the user's device.

Purpose	Source Path	Size	Type
Maskable Icon	`/web-app-manifest-192x192.png`	192x192	`image/png`
Maskable Icon	`/web-app-manifest-512x512.png`	512x512	`image/png`
Apple Home Screen	`/apple-touch-icon.png`	180x180	`image/png`

Sources: public/site.webmanifest:6-22, public/favicon-96x96.png:1-4, public/icon-96x96.png:1-4

Favicon & Asset Generation

This page documents the automated asset pipeline used to generate the full suite of favicons and web application icons for LegalOrbis. The system transforms a single

source SVG into multiple optimized formats (PNG, ICO) and handles cache-busting variants required for modern SEO and browser compatibility.

Overview

The asset generation process is centered around a TypeScript script located in `scripts/generate-favicons.ts`. This script utilizes the `Sharp` library for high-performance image processing and `to-ico` for creating multi-resolution Windows icon files.

Key Components

- **Source File**: `public/favicon.svg` public/favicon.svg:1-2
- **Pipeline Script**: `scripts/generate-favicons.ts` scripts/generate-favicons.ts:1-133
- **Type Definitions**: `types/to-ico.d.ts` types/to-ico.d.ts:1-10
- **Output Manifest**: `public/site.webmanifest` public/site.webmanifest:1-29

Implementation Detail

The Generation Pipeline

The function `generateFavicons()` in `scripts/generate-favicons.ts` executes a sequential workflow to populate the `public/` directory.

1. **Validation**: Ensures the source `favicon.svg` exists scripts/generate-favicons.ts:12-14.
2. **PNG Generation**: Resizes the SVG into standard dimensions (16, 32, 48, 96, 180, 192, 512) using `sharp` scripts/generate-favicons.ts:19-54.
3. **ICO Compilation**: Combines 16px, 32px, and 48px PNG buffers into a single multi-layer `favicon.ico` using the `to-ico` library scripts/generate-favicons.ts:56-65.
4. **Cache-Busting Variants**: Generates duplicate files with a `-v2` suffix (e.g., `favicon-v2.ico`) to allow for immediate cache invalidation during branding updates scripts/generate-favicons.ts:74-121.

Data Flow Diagram

The following diagram illustrates the transformation from the source SVG to the final distributed assets.

Asset Transformation Workflow

```
graph TD
    subgraph "Source Space"
        SVG["public/favicon.svg"]
    end

    subgraph "Code Entity Space: scripts/generate-favicons.ts"
        SHARP["sharp() Processing"]
        TOICO["toIco() Buffer Conversion"]
        GEN["generateFavicons()"]
    end

    subgraph "Output Space: public/"
        ICO["favicon.ico (Multisize)"]
        APPLE["apple-touch-icon.png (180x180)"]
        PNGS["favicon-16x16.png / 32x32.png / 48x48.png"]
        V2["v2 Cache-Busting Variants"]
        MANIFEST["site.webmanifest"]
    end

    SVG --> GEN
    GEN --> SHARP
    SHARP --> PNGS
    SHARP --> APPLE
    PNGS --> TOICO
    TOICO --> ICO
    PNGS --> V2
    MANIFEST -.-> PNGS
```

Sources: scripts/generate-favicons.ts:9-132, public/site.webmanifest:5-23

Technical Specifications

Asset Dimensions and Roles

The pipeline generates specific sizes tailored for different platforms:

File	Size	Purpose
`favicon-16x16.png`	16x16	Legacy browser tabs
`favicon-32x32.png`	32x32	Standard browser tabs
`favicon-48x48.png`	48x48	Google Search results (minimum)
`favicon-96x96.png`	96x96	Google Search results (preferred)
`apple-touch-icon.png`	180x180	iOS Home Screen
`favicon.ico`	16, 32, 48	Legacy Windows compatibility
`web-app-manifest-192.png`	192x192	PWA / Android

Sources: scripts/generate-favicons.ts:18-54, public/site.webmanifest:5-22

Type Safety

Because the `to-ico` package lacks official TypeScript definitions, a custom declaration is maintained in `types/to-ico.d.ts`. This defines the `toIco` function signature, accepting an array of Buffers and optional size configurations types/to-ico.d.ts:1-10.

Type-Asset Relationship

```
graph LR
    subgraph "types/to-ico.d.ts"
        TS_DEF["declare module 'to-ico'"]
    end

    subgraph "scripts/generate-favicons.ts"
        IMPORT["import toIco from 'to-ico'"]
        CALL["await toIco([png16, png32, png48])"]
    end

    subgraph "public/"
        ICO_FILE["favicon.ico"]
    end

    TS_DEF --> IMPORT
    IMPORT --> CALL
    CALL --> ICO_FILE
```

Sources: types/to-ico.d.ts:1-10, scripts/generate-favicons.ts:4, scripts/generate-favicons.ts:58

Web Manifest Configuration

The `public/site.webmanifest` file maps these generated assets for Progressive Web App (PWA) support. It specifies: * **Branding**: Name ("Legal Orbis Abogados") and theme colors (#1A3635) public/site.webmanifest:2-24. * **Icons**: References to the 192x192 and 512x512 PNGs, marked as "maskable" for Android UI consistency public/site.webmanifest:5-17. * **Display**: Set to "standalone" to provide a native-like experience when installed public/site.webmanifest:26.

Sources: public/site.webmanifest:1-29

CI/CD & Dependency Management

This section documents the automation infrastructure of the LegalOrbis project, covering continuous integration, automated dependency maintenance, and the browser compatibility policy.

Continuous Integration (CI)

The project utilizes GitHub Actions to ensure code quality and build stability. The workflow is defined in `.github/workflows/ci.yml` and triggers on every push or pull request targeting the `main` or `dev` branches [.github/workflows/ci.yml:3-7] [CLAUDE.md:33-34].

Workflow: `lint-and-build`

The CI pipeline runs on an `ubuntu-latest` environment and executes a sequence of steps to validate the codebase before deployment to Vercel [.github/workflows/ci.yml:10-11].

1. **Environment Setup**: Uses `actions/checkout@v4` to pull the code and `actions/setup-node@v4` to initialize Node.js version 20 [.github/workflows/ci.yml:14-20]. It leverages the built-in npm caching to speed up subsequent runs [.github/workflows/ci.yml:21-21].
2. **Dependency Installation**: Executes `npm ci` for a clean, reproducible installation based on the lockfile [.github/workflows/ci.yml:24-24].
3. **Static Analysis**: Runs `npm run lint` to execute ESLint checks [.github/workflows/ci.yml:27-27].
4. **Production Build**: Runs `npm run build` to verify that the Next.js application compiles correctly and that TypeScript types are valid [.github/workflows/ci.yml:30-30].

CI Workflow Logic

The following diagram illustrates the flow from code commit to successful validation.

CI Pipeline Architecture

```
graph TD
    subgraph "GitHub Actions Runner (Ubuntu)"
        A["git push / PR"] --> B["Checkout Code"]
        B --> C["Setup Node.js v20"]
        C --> D["npm ci (Install)"]
        D --> E["npm run lint"]
        E --> F["npm run build"]
        F --> G{{"Success"}}
    end

    subgraph "Local Development"
        H["npm install"]
        I["npm run dev"]
    end
```

Sources: - `[.github/workflows/ci.yml:1-31]()` - `[CLAUDE.md:11-19]()`

Dependency Management with Renovate

LegalOrbis uses Renovate bot to automate the process of keeping dependencies up to date. The configuration is stored in `renovate.json` and is tailored to reduce manual maintenance overhead while ensuring stability.

Configuration Strategy

- **Schedule**: Updates are restricted to Monday mornings before 9:00 AM (Europe/Madrid timezone) to minimize noise during the work week [renovate.json:4-5].
- **Automerge**: The bot is configured to automatically merge `patch` updates, assuming they contain non-breaking bug fixes [renovate.json:9-11]. `minor` and `major` updates require manual review and approval [renovate.json:13-15].
- **Grouping**: Related packages are grouped into single Pull Requests to reduce the number of notifications:
  - **Next.js Group**: Combines `next` and `eslint-config-next` [renovate.json:17-19].
  - **React Group**: Combines `react`, `react-dom`, and their corresponding TypeScript types [renovate.json:21-23].

Dependency Logic Mapping

Dependency Update Flow

```
graph LR
    subgraph "Renovate Bot"
        R1["Check Registry"] --> R2{"Update Type?"}
        R2 -- "Patch" --> R3["Apply Label: dependencies"]
        R3 --> R4["Run CI"]
        R4 --> R5["Automerge"]

        R2 -- "Minor/Major" --> R6["Group Packages"]
        R6 --> R7["Create PR"]
        R7 --> R8["Wait for Manual Review"]
    end

    subgraph "Package Groups"
        G1["Next.js Group"] --- next["next"]
        G1 --- ecn["eslint-config-next"]
        G2["React Group"] --- r1["react"]
```

```
        G2 --- r2["react-dom"]
    end
```

Sources: - `[renovate.json:1-25]()`

Browser Support Policy

The project maintains a modern browser support policy defined in `.browserslistrc`. This configuration informs tools like Tailwind CSS and the Next.js compiler (via SWC) about the target JavaScript and CSS features [.browserslistrc:1-8].

Target Environment

The policy targets modern browsers and explicitly excludes legacy environments to keep the bundle size small and avoid unnecessary polyfills: * **Coverage**: Browsers with >0.5% global usage [.browserslistrc:2-2]. * **Recency**: The last 2 versions of major browsers [.browserslistrc:3-3]. * **Exclusions**: Explicitly excludes Internet Explorer 11 and Opera Mini [.browserslistrc:5-6].

Impact on Styling

The CSS implementation relies on modern features such as CSS Custom Properties (variables) and Tailwind 4. Components like `AreaDetailHero` use modern CSS properties like `text-shadow` and `backdrop-filter` (via `blur-3xl`) which are supported by the targeted browser set [components/area-detail-hero.tsx:55,62,78-79].

Sources: - `[.browserslistrc:1-8]()` - `[components/area-detail-hero.tsx:52-81]()`

Glossary

This glossary defines the technical terminology, domain-specific legal concepts, and framework-level jargon used throughout the LegalOrbis codebase. It serves as a

reference for onboarding engineers to understand how natural language requirements translate into technical implementation.

1. Legal Domain Terminology (Spanish)

These terms represent the business logic and content categories defined in the legal data model.

Term	Technical Context	Definition
Área Jurídica	`areasData` keys	A primary practice area (e.g., Penal, Civil). Represented by the `AreaData` interface lib/data/areas-juridicas.ts:2.
Rama (Branch)	`area.branches`	Specific sub-specialties within a legal area used for UI lists lib/data/areas-juridicas.ts:44-50.
Derecho Penal	`areasData.penal`	Criminal law. The core specialty of the firm lib/data/areas-juridicas.ts:3-11.
Derecho Penitenciario	`areasData.penal`	Prison law. Included within the Penal area data lib/data/areas-juridicas.ts:5.
Investigado	FAQ content	A person under investigation in a criminal procedure; a key keyword for SEO lib/data/areas-juridicas.ts:58.

Sources: lib/data/areas-juridicas.ts:1-82

2. Codebase Specific Terms

Terms unique to the LegalOrbis architecture and implementation patterns.

Dynamic Metadata Factory

A pattern used in `lib/seo/metadata.ts` where functions like `generateAreaMetadata` act as factories to create Next.js `Metadata` objects by merging site-wide defaults with area-specific data lib/seo/metadata.ts:152-181.

Schema Injector

The system uses functions in `lib/seo/schema.ts` to generate JSON-LD objects. These are "injected" into the `<head>` of pages to provide structured data to search engines lib/seo/schema.ts:121-280.

Contact Form Reusable

A specific component pattern where the form logic is abstracted into `hooks/useContactForm.ts` to support both the standalone contact page and the Header's modal trigger hooks/useContactForm.ts:29-91.

Sources: lib/seo/metadata.ts:152-181, lib/seo/schema.ts:1-280, hooks/useContactForm.ts:29-91

---

3. Technical & Framework Jargon

Standard industry terms as they apply specifically to this project's configuration.

AVIF/WebP Optimization

The image pipeline configured in `next.config.ts` that prioritizes modern, high-compression formats for legal imagery next.config.ts:4-11.

Cache-Busting (v2)

A strategy implemented in `app/robots.ts` and `scripts/generate-favicons.ts` where assets like `favicon-v2.ico` are explicitly allowed and versioned to bypass stale browser caches app/robots.ts:14-17.

Immutable Assets

Assets served with a 1-year `Cache-Control` TTL (31,536,000 seconds) and the `immutable` flag, as configured for the `_next/static` and `/images` directories next.config.ts:80-96.

Sources: next.config.ts:4-152, app/robots.ts:9-25

---

4. Architectural Diagrams

Concept Mapping: Natural Language to Code Entities

The following diagram bridges how a user's request (e.g., "I want to see the Penal Law page") maps to specific code structures.

Title: Request Resolution Flow

```
graph TD
    UserRequest["User clicks 'Derecho Penal'"] --> Slug["Slug: 'penal'"]
    Slug --> DataLookup["areasData['penal'] in lib/data/areas-juridicas.ts"]

    subgraph "Metadata Generation"
        DataLookup --> MetaFactory["generateAreaMetadata() in lib/seo/metadata.ts"]
        MetaFactory --> NextMeta["Next.js Metadata Object"]
    end

    subgraph "Schema Injection"
        DataLookup --> SchemaFactory["generateLegalServiceSchema() in lib/seo/schema.ts"]
        SchemaFactory --> JSONLD["LD+JSON Script Tag"]
    end

    subgraph "UI Composition"
        DataLookup --> PageComp["app/areas-juridicas/[slug]/page.tsx"]
        PageComp --> Hero["AreaDetailHero Component"]
        PageComp --> FAQ["AreaFAQ Component"]
    end
```

Sources: lib/data/areas-juridicas.ts:2-79, lib/seo/metadata.ts:152-181, lib/seo/schema.ts:234-262

Data Flow: Form Submission

This diagram traces the path of data from the user interface to the external service provider.

Title: Contact Form Data Pipeline

```
graph LR
    UI["ContactForm Component"] -- "onChange" --> Hook["useContactForm.ts"]
    Hook -- "state: ContactFormData" --> Validation["HTML5 Validation"]
    Validation -- "onSubmit" --> FetchCall["fetch('https://formspree.io/f/manlbdwz')"]

    subgraph "Internal State"
        Hook
    end

    subgraph "External"
        FetchCall --> Formspree["Formspree API"]
        Formspree --> Email["Firm Inbox (info@...)"]
    end
```

Sources: hooks/useContactForm.ts:3-91, components/contact-form.tsx

---

5. Abbreviations & Keys

Abbreviation	Full Term	Context
CVA	Class Variance Authority	Used in `components/ui/` for managing Tailwind variants.
JSON-LD	JSON for Linked Data	The format used in `lib/seo/schema.ts` for SEO.
TTL	Time To Live	Cache duration configured in `next.config.ts` next.config.ts:8.
OG	Open Graph	Metadata for social sharing (Facebook, LinkedIn) lib/seo/metadata.ts:52.
CSP		

Abbreviation	Full Term	Context
	Content Security Policy	Security headers for SVG handling next.config.ts:10.

Sources: next.config.ts:1-205, lib/seo/metadata.ts:1-181